

Computer Vision for Automated Driving

Elias W.J. Müller

FS26

Contents

1	Part I: Fundamentals	3
1.1	Definitions	3
1.2	Optical Autonomous Driving Sensors	3
1.2.1	Light Detection and Ranging (Lidar)	4
1.2.2	Camera	6
1.2.3	Radar	7
1.2.4	Sensor Distribution on Car	7
2	Part II: Semantic Perception	8
2.1	Semantic Segmentation	8
2.1.1	Definition and Basics	8
2.1.2	Exemplar Driving Datasets	8
2.1.3	Convolutional Neural Networks Primer	8
2.1.4	Encoder–Decoder Paradigm	10
2.1.5	Fundamental Architectures and Modules	11
2.2	Object Detection	12
2.2.1	Fundamentals and Definitions	12
2.2.2	Two Stage R-CNN	13
2.2.3	Direct Detectors	15
2.3	Instance Segmentation	16
2.3.1	Mask R-CNN	16
2.4	Panoptic Segmentation	17
2.4.1	Fundamentals and Definitions	17
2.4.2	Instance-semantic Framework for Panoptic Segmentation	17
2.4.3	Unified Mask-based Framework for Segmentation	18
3	Part III: Geometric Perception and Localization	21
3.1	Depth and 3D Estimation	21
3.1.1	Supervised Methods	21
3.1.2	Stereo Depth Estimation	22
3.1.3	Self-Supervised Monocular Depth Estimation from Stereo	24
3.1.4	Self-Supervised Depth and Motion Estimation from Video	24
3.2	3D Classification, Segmentation, and Detection	25
3.2.1	Foundations and Definitions	25
3.2.2	3D Classification and Semantic Segmentation on Point Clouds	25
3.2.3	3D Object Detection on Point Clouds	27
3.3	3D Reconstruction and Localization	29
3.3.1	Motion Estimation from 3D and ICP Algorithm	29
3.3.2	Motion Estimation from 2D and VO	30
3.3.3	Epipolar Geometry, Two-frame SFM, and the 8-Point Algorithm	31
3.3.4	RANSAC	32
3.3.5	Learned Feature Mapping and SuperGlue	33
3.3.6	Neural Radiance Fields (NeRF)	34

4	Part IV: Robust Perception: Multi-Modal, Multi-Domain and Multi-Agent Methods	36
4.1	Domain Adaptation	36
4.1.1	Motivation, Taxonomy, and Fundamentals	36
4.1.2	Input-Level Adaptation	36
4.1.3	Feature-Level Adaptation	39
4.1.4	Output-Level Adaptation	39
4.1.5	Test-Time Domain Adaptation	41
4.2	Sensor Fusion	41
4.2.1	Basics and Taxonomy	42
4.2.2	Lidar-Camera Fusion for 3D Object Detection	42
4.2.3	Radar-Camera Fusion for 3D Object Detection	45
4.2.4	Multi-Model 2D Object Detection	46
4.3	Visual Grounding, Anomaly Segmentation, and Vehicle-to-Vehicle Communication	47
4.3.1	Visual Grounding	47
4.3.2	Anomaly Segmentation	48
4.3.3	Vehicle-to-Vehicle Communication for Perception	49
5	Part V: Temporal Perception	50
5.1	Multiple Object Tracking	50
5.1.1	Tracking by Detection (Classical)	50
5.1.2	End-to-End MOT with Transformers (MOTR)	51
5.2	Motion Prediction	51

1 Part I: Fundamentals

1.1 Definitions

Definition 1 (Computer Vision). *Inverse algorithmic process to infer properties of the world and of optical sensors based on sensory observations.*

Definition 2 (Autonomous Car (ADS-DV)). *ADS-equipped vehicle designed for driverless operation within its given operational design domain.*

Definition 3 (Automated Driving System (ADS)). *Hardware and software collectively capable of performing the entire DDT on a sustained basis, possibly limited to a specific ODD.*

Definition 4 (Operational Design Domain (ODD)). *Conditions (environmental, geographical, temporal, traffic/roadway) under which a given ADS is designed to function.*

Definition 5 (Dynamic Driving Task (DDT)). *All real-time operational and tactical functions for on-road driving (excludes strategic functions). Subtasks: (1) lateral control, (2) longitudinal control, (3) OEDR monitoring, (4) OEDR execution, (5) maneuver planning, (6) conspicuity enhancement.*

Definition 6 (OEDR). *Object and Event Detection and Response. DDT subtasks 3–4: monitor driving environment via detection, recognition, classification, response preparation, and response execution.*

Definition 7 (Driving Automation System). *Hardware and software capable of performing part or all of the DDT. $ADS \subseteq$ Driving Automation Systems. Four core modules: (1) semantic and geometric **Perception** (2) **Prediction** from temporal perception (3) path and motion **Planning** via response determination (4) **Control***

Level	Name	Narrative definition	Execution of steering and acceleration/deceleration	Monitoring of driving environment	Fallback performance of dynamic driving task	System capability (driving modes)
Human driver monitors the driving environment						
0	No Automation	the full-time performance by the human driver of all aspects of the dynamic driving task, even when enhanced by warning or intervention systems	Human driver	Human driver	Human driver	n/a
1	Driver Assistance	the driving mode-specific execution by a driver assistance system of either steering or acceleration/deceleration using information about the driving environment and with the expectation that the human driver perform all remaining aspects of the dynamic driving task	Human driver and system	Human driver	Human driver	Some driving modes
2	Partial Automation	the driving mode-specific execution by one or more driver assistance systems of both steering and acceleration/deceleration using information about the driving environment and with the expectation that the human driver perform all remaining aspects of the dynamic driving task	System	Human driver	Human driver	Some driving modes
Automated driving system ("system") monitors the driving environment						
3	Conditional Automation	the driving mode-specific performance by an automated driving system of all aspects of the dynamic driving task with the expectation that the human driver will respond appropriately to a request to intervene	System	System	Human driver	Some driving modes
4	High Automation	the driving mode-specific performance by an automated driving system of all aspects of the dynamic driving task, even if a human driver does not respond appropriately to a request to intervene	System	System	System	Some driving modes
5	Full Automation	the full-time performance by an automated driving system of all aspects of the dynamic driving task under all roadway and environmental conditions that can be managed by a human driver	System	System	System	All driving modes

Figure 1: SAE levels of driving automation

1.2 Optical Autonomous Driving Sensors

Countless sensor choices exist. We make the distinction of **external** (camera, lidar, radar, event camera, GPS, IMU, speedometer, ...) and **internal** (camera, thermal camera, microphone) sensors. Our focus further lies on the *Optical Sensors*.

Optical Sensors can be categorized along the following features:

- *Wavelength*: spectral band of light the sensor operates in (e.g. visible, near-infrared, millimeter-wave).
- *Active or Passive*: whether the sensor emits its own signal (active) or relies on external illumination (passive).
- *Resolution*: spatial density of measurements; determines fineness of detail captured.
- *Latency / Frame Rate*: temporal frequency at which full measurements are acquired.
- *Field of View* (FOV): angular extent of the observable scene, horizontal and vertical.
- *Range*: maximum distance at which the sensor can reliably detect objects.

1.2.1 Light Detection and Ranging (Lidar)

Characteristics: Active sensor, near-infrared wavelength (905nm), direct range measurements via time of flight (TOF), i.e., timing of light emission until recapture of photons, large horizontal and fair vertical FOV (3D), fair resolution

Pulsed Lidar

Multiple short laser pulses emitted in different directions and at different times. Each measurement: pulse emitter → beam steering → target → photodetector (APD). A timer measures the round-trip delay to infer range R_0

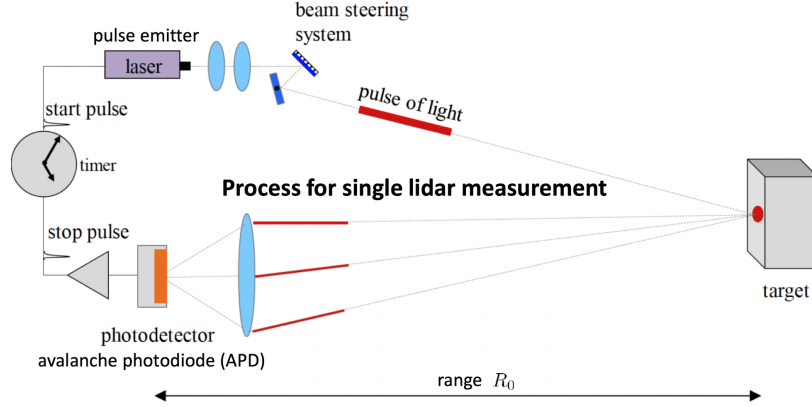


Figure 2: Lidar working principle

Beam Configurations.

Real beams diverge; receivers have finite FOV. Two configurations:

- *Coaxial*: transmitter and receiver share a common optical axis.
- *Bistatic*: optical axes of transmitter and receiver are parallel but offset. The **crossover function** $\xi(R) \in [0, 1]$ gives the fraction of the illuminated area observed by the receiver as a function of range. For bistatic: $\xi(R) = 0$ for $R < R_1$ (no crossover), $\xi(R) \in (0, 1)$ for $R_1 < R < R_2$ (partial crossover), $\xi(R) = 1$ for $R > R_2$ (full crossover).

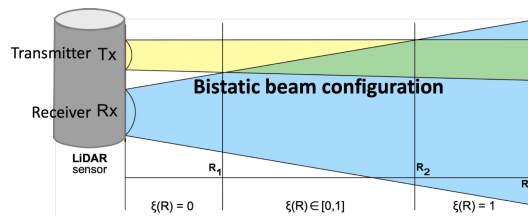


Figure 3: Lidar Bistatic Beam Configuration

Pulse Waveform (Sinusoidal Form).

$$P_T(t) = \begin{cases} P_0 \sin^2\left(\frac{\pi}{2\tau_H} t\right), & 0 \leq t \leq 2\tau_H, \\ 0, & \text{otherwise.} \end{cases}$$

where $P_T(t)$: emitted radiant power, P_0 : peak power, τ_H : half-power pulse width (10–20 ns), $c\tau_H$: spatial half-power width (3–6 m).

Physically more realistic is the square-exponential form (left heavy, right tailed) – omitted here.

Linear System Model.

Received radiant power as a convolution of emitted power with the optical system impulse response:

$$P_R(R) = C_A \int_0^{2R/c} P_T(t) H\left(R - \frac{ct}{2}\right) dt$$

where $P_R(R)$: range-dependent received power, $H(R)$: impulse response of the optical system, C_A : system constant (independent of time and range). $ct/2$ maps time t to the corresponding range (round-trip).

Intuition: $H(R)$ is what we would receive from a perfectly concentrated (Dirac) pulse; the actual response is a smeared version due to finite pulse width - ambiguity from temporal profile/ early or late photon.

Impulse Response Decomposition.

$$H(R) = H_C(R) H_T(R)$$

- *Optical channel* (medium): $H_C(R) = \frac{T^2(R)}{R^2} \xi(R)$, where $T(R) = \exp(-\alpha R)$ is the one-way transmission loss (α depends on visibility, assumes homogeneous scattering medium). Similar to what we see in NeRF or Fog Simulation, but squared, since light must travel back and forth.
- *Target*: $H_T(R) = \rho_0 \delta(R - R_0)$ in clear weather, where ρ_0 is the differential reflectivity.

Clear Weather Simplification.

With $\alpha = 0$ (no attenuation – clear air) and $\xi(R_0) = 1$:

$$H(R) = \frac{1}{R_0^2} \rho_0 \delta(R - R_0)$$

Substituting into the linear system model yields:

$$P_R(R) = \begin{cases} \frac{C_A P_0 \rho_0}{R_0^2} \sin^2\left(\frac{\pi(R - R_0)}{c\tau_H}\right), & R_0 \leq R \leq R_0 + c\tau_H, \\ 0, & \text{otherwise.} \end{cases}$$

Intuition: The target acts as a Dirac delta in range, so the convolution merely shifts and scales the emitted waveform — the received pulse preserves the shape of P_T , centered at R_0 . The $1/R_0^2$ decay is the inverse-square law: emitted power spreads over a sphere of area $\propto R_0^2$. The proportionality to ρ_0 is why lidar returns per-point *intensity* alongside range. The pulse occupies a range interval of width $c\tau_H$, so shorter pulses yield finer range resolution at the cost of less energy per pulse.

Features of Pulsed Lidar.

- *Ranging precision*: limited by temporal resolution of timing electronics. Typical $\sim 0.1 \text{ ns} \Rightarrow \sim 1.5 \text{ cm}$.
- *Maximum range*: limited by signal to noise ratio (SNR) (attenuation + detector noise/jitter): 100–200 m.
- *Range-rate trade-off*: maximum unambiguous range is inversely proportional to pulse repetition rate (aliasing) – upper limit of the integral

Acquisition of 3D Point Clouds.

Each pulse yields a single range measurement along a direction (α, ϵ) . To obtain a 3D point cloud, the sensor scans across a range of azimuth α and elevation ϵ values, which jointly define the FOV. Spherical-to-Cartesian conversion:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{h}^{-1}(r, \alpha, \epsilon) = \begin{bmatrix} r \cos \alpha \cos \epsilon \\ r \sin \alpha \cos \epsilon \\ r \sin \epsilon \end{bmatrix}$$

Each point additionally carries an *intensity* value $= f(P_R(R_0))$, derived from the peak received power.

Scanning Mechanisms.

Taxonomy: non-scanning (flash) vs. scanning lidar. Scanning subdivides into non-mechanical (OPA, solid-state) and mechanical (MEMS, motorized optomechanical). Driving-relevant types are motorized mechanical and MEMS scanners.

Motorized mechanical: nodding-mirror tilt controls vertical FOV; rotating housing ($> 1000 \text{ RPM}$) provides panoramic 360° horizontal FOV. Modern systems use laser/photodiode arrays $\Rightarrow$ mechanical rotation only along azimuth. Horizontal angular resolution up to 0.1° , max range up to 200 m.

MEMS: tiny oscillating mirrors (few mm diameter) steer individual pulses. 1D MEMS scans one angular dimension (needs laser array or motor for the other); 2D MEMS scans both. Resonant scanning: frequency = resonant frequency of mirror, high frame rate, sensitive to environmental variation. Non-resonant: more robust, lower frame rate.

Motorized Mechanical	MEMS
Panoramic 360° horizontal FOV	Limited ($\sim 120^\circ$) horizontal FOV
Uniform azimuthal scanning pattern	Complex / less interpretable pattern
Bulky, vulnerable to wear	Small, no mechanical wear
Imbalanced vert. vs. horiz. resolution	Balanced resolution
Low frame rate (10–20 Hz)	Slightly higher frame rate
Expensive	Less expensive

Measurement Errors in Lidar.

- *Sensor*: timing jitter in arrival detection; rotary encoder uncertainty (angular error $\rightarrow$ distortion).
- *Scene*: range ambiguity (multiple waveform maxima); non-Lambertian surfaces (absorptive $\rightarrow$ no return, specular $\rightarrow$ multi-path); motion distortion from finite frame rate (not rectifiable, unlike ego-motion).
- *Optical medium*: attenuation (e.g. fog $\rightarrow$ reduced peak amplitude); backscattering (e.g. snow/rain $\rightarrow$ false peaks, spurious points).
- *Light sources*: ambient NIR interference (minor); cross-talk from other lidars.

1.2.2 Camera

Characteristics: Passive sensor, visible spectrum (typically three sensitivity profiles / channels), very high resolution, fair frame rate, moderate FOV for perspective cameras, cheap.

Pinhole Camera Model.

Ideal pinhole = center of projection $\mathbf{O}_c$. All scene rays pass through a single point onto the image plane. Virtual image plane (in front of pinhole): image is not inverted. This yields a *perspective projection*.

Homogeneous Coordinates.

Unify representation of points and planes ($ax + by + cz - d = 0$) by adding one dimension:

$$\mathbf{P} = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \quad (\text{point}), \quad \mathbf{\Pi} = \begin{pmatrix} a \\ b \\ c \\ -d \end{pmatrix} \quad (\text{plane}),$$

where $\mathbf{\Pi} \cdot \mathbf{P} = 0$ iff $\mathbf{P}$ lies on $\mathbf{\Pi}$.

Geometric Transformations.

Notation: ${}^F\mathbf{P}$ denotes coordinates of point $\mathbf{P}$ in frame F .

- *Translation*: ${}^B\mathbf{P} = {}^A\mathbf{P} + {}^B\mathbf{O}_A$, where ${}^B\mathbf{O}_A$ is the origin of frame A expressed in frame B .
- *Rotation*: ${}^B\mathbf{P} = {}^B_A\mathbf{R} {}^A\mathbf{P}$, where ${}^B_A\mathbf{R} = ({}^B\mathbf{i}_A \ {}^B\mathbf{j}_A \ {}^B\mathbf{k}_A)$ and $\mathbf{R}^T\mathbf{R} = \mathbf{I}$.
- *Rigid / Euclidean*: ${}^B\mathbf{P} = {}^B_A\mathbf{R} {}^A\mathbf{P} + {}^B\mathbf{O}_A$. In homogeneous coordinates:

$$\begin{bmatrix} {}^B\mathbf{P} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^B_A\mathbf{R} & {}^B\mathbf{O}_A \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} {}^A\mathbf{P} \\ 1 \end{bmatrix}$$

3D to 2D Perspective Projection.

Given a 3D point $\mathbf{P}_c = (x_c, y_c, z_c)^T$ in camera coordinates, projection to 2D homogeneous pixel coordinates $\tilde{\mathbf{x}}_s$:

$$\tilde{\mathbf{x}}_s \sim \mathbf{K} \mathbf{P}_c, \quad \mathbf{K} = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where f_x, f_y : focal lengths, s : skew, (c_x, c_y) : principal point, $\mathbf{K}$ encoding the camera intrinsics (5 DOF)

Intuition: Derived from similar triangles starting with: $-(x_s - c_x)/x_c = f/(-z_c)$.

Incorporating Extrinsic. rigid transformation from world frame to camera frame: $\mathbf{P}_c = [\mathbf{R} \mid \mathbf{t}] \tilde{\mathbf{P}}_w$

Complete Projection.

from 3D homogeneous world coordinates to 2D homogeneous image coordinates:

$$\tilde{\mathbf{x}}_s \sim \underbrace{\mathbf{K}[\mathbf{R} \mid \mathbf{t}]}_{\mathbf{C}} \tilde{\mathbf{P}}_w \quad \mathbf{C}: 3 \times 4 \text{ camera matrix (11 DOF: 5 intrinsics + 6 extrinsics)}$$

Photometric Image Formation.

Digital imaging pipeline (scene radiance $\rightarrow$ optics $\rightarrow$ sensor $\rightarrow$ ISP $\rightarrow$ output), BRDF-based (Bidirectional Reflectance Distribution Function) rendering equation, Lambertian vs. specular reflection. (details omitted)

1.2.3 Radar

Characteristics: Active sensor, 70–80 GHz (wavelengths $\sim$ few mm), measures *range and velocity*. Robust to fog/rain/snow (wavelength $\gg$ particle size $\Rightarrow$ no scattering). Large horizontal FOV, very narrow vertical FOV. Low azimuthal resolution (few azimuth bins across the 360° sweep), high range resolution. Low frame rate. Range: 200–600 m.

FMCW Scanning Radar.

Two waveform types: fixed-frequency pulses and frequency-modulated continuous wave (FMCW). Output is discretized into M azimuths $\times$ N range bins; per-bin measurement is received power.

Radar equation: $P_{\text{received}} = \frac{P_{\text{transmitted}} G \sigma A_e}{16\pi^2 r^4 L}$ where G : transmitter gain, σ : radar cross section (RCS, depends on target area/orientation/material), A_e : effective antenna aperture, L : loss factor, r : range. Note $1/r^4$ decay (vs. $1/R_0^2$ for lidar): the wave spreads $\propto 1/r^2$ out to the target and the reflected wave spreads a further $\propto 1/r^2$ back to the receive aperture (two-way spreading). This is the radar analogue of the pulsed lidar received-power model: it determines how SNR degrades with range and target properties, and thus the sensor’s effective detection limits.

FMCW Range and Velocity Measurement:

Modulation pattern: frequency of transmitted signal as a function of time.

Intermediate frequency (IF): difference between transmitted and received signal frequencies.

For a *static target*, a single linear frequency sweep (chirp, i.e. frequency ramping at constant rate df/dt) yields range directly from the IF: $r = \frac{c\Delta f}{2(df/dt)}$ where Δf is the IF.

For a *moving target*, there is ambiguity between TOF shift and Doppler shift Δf_v . Resolved using repeating triangular modulation pattern (up-chirp IF = Δf_1 , down-chirp IF = Δf_2):

$$\Delta f_v = \frac{\Delta f_2 - \Delta f_1}{2}, \quad \Delta f_r = \frac{\Delta f_1 + \Delta f_2}{2} \quad (1)$$

Δf_r gives range; Δf_v gives radial velocity relative to sensor.

Range-FFT: FFT of the IF signal yields the power-range spectrum for each azimuth. Each frequency component in the IF corresponds to a specific range, so the FFT separates targets at different distances. Real targets are not point-like $\Rightarrow$ energy spreads across multiple range bins, unlike the ideal single-peak response.

1.2.4 Sensor Distribution on Car

Typical ADS platform uses 20–40 sensors. Sensor placement affects covered FOV, cost, reliability, dirt management, serviceability, and crash survivability. Roof-mounted setups (common for robotaxis / data collection) are not acceptable for commercial vehicles. Changes in sensor 6D pose introduce dataset bias, potentially requiring full re-collection. Sensor suite composition is still under active definition in the industry.

2 Part II: Semantic Perception

2.1 Semantic Segmentation

2.1.1 Definition and Basics

Definition 8 (Semantic Image Segmentation). *Given a predefined class set $\mathcal{C} = \{1, \dots, C\}$, assign every pixel $\mathbf{x}_i$ of an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ a single class label $y_i \in \mathcal{C}$. No differentiation between distinct instances of the same class (cf. instance/panoptic segmentation).*

Data Format.

Supervised setup: dataset $\mathcal{D} = \{(\mathbf{I}^{(n)}, \mathbf{Y}^{(n)})\}_{n=1}^N$.

- *Input*: RGB image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$.
- *Output / ground truth*: one-hot tensor $\mathbf{Y} \in \{0, 1\}^{H \times W \times C}$ with $\sum_{c=1}^C Y_{h,w,c} = 1$ for every pixel (h, w) . One channel per class.

Indexing convention: $h \in \{1, \dots, H\}$ row, $w \in \{1, \dots, W\}$ column, $c \in \{1, \dots, C\}$ class channel, k used as a dummy class index in sums.

Prediction Format.

Model outputs a logits tensor $\mathbf{Z} \in \mathbb{R}^{H \times W \times C}$. Per-pixel softmax gives the soft prediction tensor $\hat{\mathbf{Y}} \in [0, 1]^{H \times W \times C}$:

$$\hat{Y}_{h,w,c} = \frac{\exp(Z_{h,w,c})}{\sum_{k=1}^C \exp(Z_{h,w,k})}, \quad \hat{\mathbf{Y}}_{h,w,:} \in \Delta^{C-1}, \quad \text{Hard prediction: } \hat{y}_{h,w} = \arg \max_c \hat{Y}_{h,w,c}.$$

Categorical Cross-Entropy Loss.

$$\mathcal{L}(\mathbf{I}) = -\frac{1}{HW} \sum_{h=1}^H \sum_{w=1}^W \sum_{c=1}^C Y_{h,w,c} \log \hat{Y}_{h,w,c}.$$

Intuition: One-hot $\mathbf{Y}$ collapses inner c -sum to the single true-class term ($\log \hat{Y}_{h,w,y_{h,w}}$) $\rightarrow$ average over HW pixels.

Evaluation Metrics.

- *Pixel accuracy*: fraction of correctly classified pixels. Skewed by frequent classes (sky, road).
- *IoU (class c)*: $\text{IoU}_c = \frac{|\hat{P}_c \cap P_c|}{|\hat{P}_c \cup P_c|}$, with $\hat{P}_c, P_c$ the predicted/GT pixel sets.
- *mIoU*: $\frac{1}{C} \sum_c \text{IoU}_c$. More stringent than pixel accuracy; mean over classes prevents frequent/large classes (sky, road) from dominating the score, unlike pixel accuracy.

2.1.2 Exemplar Driving Datasets

Dataset	Scale / resolution	Notes
Cityscapes	5k fine + 20k coarse, 1024×2048	30 classes / 8 categories; SOTA mIoU > 85%
Mapillary Vistas	$4000^+ \times 3000^+$	66 classes (37 instance-level); global, multi-viewpoint
Foggy Cityscapes	inherits Cityscapes labels	fog simulation at varying densities
GTA5	24,966 frames, HR	label propagation via rendering pipeline (~ 7 s/img)
ACDC	4006 adverse-cond. images	fog/night/rain/snow + clear correspondences
ApolloScape	140k pixel-level annot.	dense traffic
IDD	unstructured environments	unconventional classes, high intra-class diversity
WildDash	evaluation/testing	organized by visual hazards

2.1.3 Convolutional Neural Networks Primer

Limitations of MLPs on Images.

Vectorizing an $H \times W$ image as MLP input

1. destroys 2D structure and local correlations; spatial invariances must be relearned;
2. explodes the parameter count (e.g. 400×250 image, 10k hidden $\Rightarrow \sim 10^9$ parameters in the first layer alone).

Discrete 2D Convolution.

Slide the kernel $\mathbf{K} \in \mathbb{R}^{m \times n}$ over input $\mathbf{X}$; at each location multiply the overlapping pixel-weight pairs and sum:

$$(\mathbf{X} * \mathbf{K})[p, q] = \sum_{i=1}^m \sum_{j=1}^n X[p+i-1, q+j-1] K[i, j].$$

Formally a *cross-correlation*; it differs from convolution proper by a kernel flip.

Intuition: The operation is *local*, *linear*, and *shift-equivariant* (commutes with translation). The same parameters are used across all spatial locations – *parameter sharing*. The 2D structure of the input is preserved in the output.

Convolutional Layer: General Formulation.

At layer ℓ , the pre-activation tensor of the previous layer

$$\mathbf{z}_{\ell-1} \in \mathbb{R}^{H_{\ell-1} \times W_{\ell-1} \times d_{\ell-1}}$$

is convolved (after element-wise activation g) with d_ℓ 3D kernels

$$\mathbf{W}^{(\ell, c)} \in \mathbb{R}^{m_\ell \times n_\ell \times d_{\ell-1}}, \quad c = 1, \dots, d_\ell,$$

each paired with a scalar bias $b_c^{(\ell)}$. One distinct kernel channel exists for every input channel; the kernel is therefore a 3D tensor whose depth matches $d_{\ell-1}$.

Single Convolutional Neuron.

For output channel c at spatial location (p, q) , the pre-activation of layer ℓ is the sum of 2D cross-correlations against the activated outputs of the previous layer, plus the bias:

$$z_{\ell, c}[p, q] = \sum_{k=1}^{d_{\ell-1}} \sum_{i=1}^{m_\ell} \sum_{j=1}^{n_\ell} w_{k, c}^{(\ell)}[i, j] g(z_{\ell-1, k}[p+i-1, q+j-1]) + b_c^{(\ell)},$$

where $g(\cdot)$ is the (non-linear) activation function and $d_{\ell-1}$ denotes the number of input channels (i.e. the output channels of layer $\ell-1$, a hyperparameter of the preceding layer); m_ℓ, n_ℓ are the kernel's spatial dimensions.

Intuition: The 2D cross-correlation generalizes to multi-channel inputs as a triple sum (two spatial + one channel dim); the 3D kernel “stacks” the channel-wise correlations and the bias offsets the result. Parameters are shared across all spatial neurons of the same output channel.

Element-wise Nonlinearity.

The activation g is applied independently to every scalar entry of the pre-activation tensor $\mathbf{z}_\ell$; the activated value

$$g(z_{\ell, c}[p, q])$$

is what enters the next layer's convolution. The conv-and-bias step and the nonlinearity are conceptually separate sub-layers; g couples neither spatial positions nor channels.

Output Tensor.

Stacking the d_ℓ feature maps gives the 3D pre-activation tensor

$$\mathbf{z}_\ell \in \mathbb{R}^{H_\ell \times W_\ell \times d_\ell},$$

i.e. multiple features per spatial location, one per output channel. d_ℓ is a layer hyperparameter

Parameter Count.

Per output channel: $m_\ell n_\ell d_{\ell-1}$ kernel weights + 1 bias. Total parameters in layer ℓ :

$$\#\theta^{(\ell)} = d_\ell (m_\ell n_\ell d_{\ell-1} + 1).$$

Intuition: Cost scales with kernel footprint and the product of input/output channels, *not* with the spatial size of the input – this is what makes convolutional layers tractable on high-resolution images.

Pooling Layer.

Non-parametric, (sub)differentiable downsampling. Hyperparameters: window size, stride. Max pooling provides robustness to local input variations and enables higher feature dimensionality without spatial-memory explosion.

Practical Variations.

- *Padding*: valid convolution shrinks spatial dims. *Same* mode pads by $\lfloor m/2 \rfloor$ (odd m) to preserve them. Modes: zero, replicate, reflect.
- *Strided convolution*: kernel slides with step $s > 1$. Motivation: faster receptive-field growth, spatial down-sampling integrated into the conv. Often in early layers but pooling more common for downsampling. Higher stride $\rightarrow$ faster increase of receptive field size

Output spatial size (per dim):

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - m}{s} \right\rfloor + 1,$$

where H_{in} : input size, m : kernel size (per dim), P : padding width per border, s : stride.

Receptive Field.

Region of the input on which a neuron's output depends. Grows with depth, kernel size, and stride. Deep CNN layers eventually attain global receptive fields covering the entire image.

CNN for Image Classification.

Architecture: (conv + activation + pool)^L $\rightarrow$ flatten $\rightarrow$ FC layers $\rightarrow$ softmax. Final FC layers learn global interdependencies needed for the image-level prediction.

2.1.4 Encoder–Decoder Paradigm

Naive: Sliding-Window Classification.

Extract a patch around each pixel, classify its center with a CNN.

Problems: massive redundant computation; no global context (esp. for large objects). Does not scale to HR images.

End-to-End Architectures.

Single forward pass from $\mathbf{I}$ to full label map $\hat{\mathbf{Y}}$. Goal: share computation across all spatial locations. Strategy: adapt classification CNNs to dense prediction.

Three Challenges.

(I) *Reduced feature resolution* from repeated striding/pooling, (II) *Multi-scale objects*, (III) *Reduced localization accuracy* from CNN invariance

Fully Convolutional Networks (FCN).

Replace the FC layers of a classification network with convolutional layers $\Rightarrow$ coarse high-level feature maps at low resolution. Per-pixel prediction at original image resolution then requires upsampling.

Note: *No single* modification is strictly necessary in every case to repurpose a classification CNN for segmentation – removing all pooling, replacing strided convs with stride-1, or adding one unpooling layer per pooling layer are each *options*, none mandatory (resolution can be recovered by other means).

Naive Resolution Recovery.

Remove pooling/striding entirely. Requires larger kernels for the same receptive field; every feature map $\mathbf{z}_\ell$ stays at HW instead of shrinking geometrically, so activation memory $\sum_\ell H_\ell W_\ell d_\ell = HW \sum_\ell d_\ell$ scales linearly with depth (no geometric decay), and the same factor inflates per-layer FLOPs $\Rightarrow$ heavy compute/memory cost and overfitting risk.

Encoder–Decoder.

- *Encoder*: maps $\mathbf{I}$ to a low-resolution, high-dimensional feature tensor; aggregates context.
- *Decoder*: upsamples encoder features back to image resolution while reducing feature dimensionality.
- Spatial resolution contracts inside the encoder (high $\rightarrow$ low) and is recovered by the decoder (low $\rightarrow$ high); feature dimensionality follows the opposite trajectory.

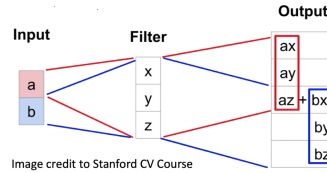
In-Network Upsampling.

Unpooling.

During encoder max-pooling, store argmax positions in *switch variables*; in the decoder, place each activation back at its recorded location. Produces an enlarged but *sparse* feature map. Not learnable + symmetric architecture for corresponding pairs.

Transpose Convolution (Deconvolution).

Learnable upsampling for dense feature maps. Each input scalar weights a copy of the kernel; the weighted kernel is placed into the output at a location determined by the input position and the stride. Where output positions overlap across different input pixels, the contributions are summed. (Crop might be required to get exact target size.) Output spatial size (per dim): $H_{\text{out}} = (H_{\text{in}} - 1)s + m - 2P$, so *increasing the stride* or *decreasing the padding* (or increasing the kernel size) all *increase* the output.

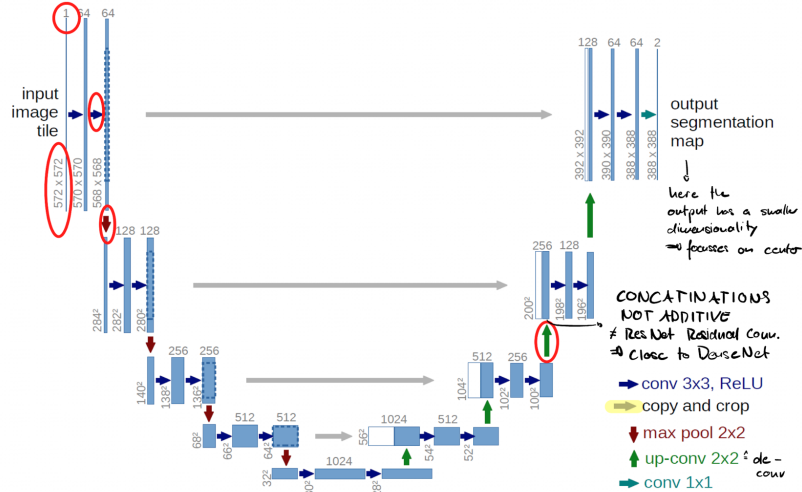


2.1.5 Fundamental Architectures and Modules

U-Net.

Symmetric encoder-decoder with *skip connections*: at each decoder stage, high-resolution encoder features are concatenated with upsampled decoder features along the channel dim, addressing the resolution challenge with fine grained localization information, followed by convolutions.

- Contracting path: $\downarrow$ “Where”, $\uparrow$ “What” (context).
- Expansive path: transpose convolutions + concatenations recover spatial detail.



Atrous (Dilated) Convolution.

The *effective* receptive field is smaller than the formal one: border connections are sparse. Atrous convolution inserts holes between kernel taps. For kernel $\mathbf{K} \in \mathbb{R}^{m \times n}$ and rate r :

$$(\mathbf{X} *_r \mathbf{K})[p, q] = \sum_{i=1}^m \sum_{j=1}^n X[p + r(i-1), q + r(j-1)] K[i, j].$$

Standard convolution corresponds to $r = 1$. Stacking atrous convolutions with growing r yields *exponential* expansion of receptive field size without resolution loss or coverage gaps and fixed number of weights. We use 0-inserts to retain tensor calculations. Changing *only* the rate r alters **only the receptive field**: the number of parameters, the FLOPs, and the stride are all unchanged.

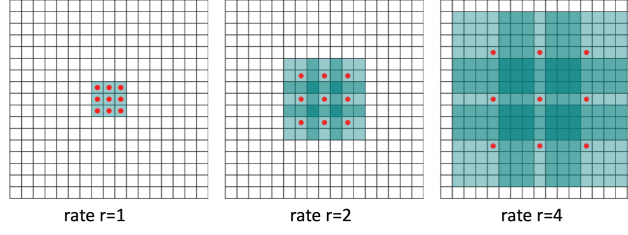


Figure 4: Atrous convolution (red dots: kernel taps)

Stride vs Atrous.

Stride downsamples the output to grow receptive field; atrous spreads the kernel to grow receptive field without downsampling. That's why atrous is preferred for dense prediction (segmentation) — you keep spatial detail.

Attention.

Provides a *variable visual receptive field*. For tokens $\mathbf{X} \in \mathbb{R}^{N \times d}$, learned projections $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ produce

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V.$$

- *Query*: representation of the token used to compare it with other tokens.
- *Key*: reference representation by which other tokens are compared against the token.
- *Value*: output representation contributed by the token when it gets “indexed”.

Scaled dot-product attention:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Softmax taken along rows (per-query distribution). Division by $\sqrt{d_k}$ keeps gradient magnitudes well-scaled. Complexity: $\mathcal{O}(N^2d)$ – quadratic in tokens.

Intuition: Query/key match measures relevance of every other token; the row-softmax weights aggregate values $\Rightarrow$ each token is a learned linear mixture of all others. Globally connected receptive field at every layer.

Transformer Block.

Multi-head self-attention (concatenate h parallel attention heads $\rightarrow$ linear projection) followed by a position-wise MLP. Residual connections and layer normalization around each sub-layer.

Transformers for Dense Prediction (Computer Vision).

ViT-style large patches lose spatial precision $\Rightarrow$ poor for segmentation. Naive fix: restrict attention to local windows of $M \times M$ patches. Complexity becomes *linear* in the number of patches (for fixed M). Drawback: receptive field bounded by the window across all layers.

Swin Transformer.

Shifted-window attention: alternate between W-MSA (window-based multi-head self-attention) and SW-MSA (shifted-window MSA). The window grid in SW-MSA is offset relative to W-MSA, so a query token's receptive field at layer $\ell + 1$ is the union of layer- ℓ windows that overlap with the token's current local window.

Intuition: With enough shifts, local windowed attention reaches large receptive fields, keeping per-layer cost linear.

Hierarchical Structure.

Patch merging concatenates features of each 2×2 neighboring patch group and applies a linear projection: tokens reduce by $4 \times$ ($2 \times$ spatial downsampling), output dim set to $2d$. Yields a multi-scale feature pyramid; Swin can serve as a drop-in CNN-backbone replacement for the encoder of standard segmentation networks.

2.2 Object Detection

2.2.1 Fundamentals and Definitions

Definition 9 (Object Detection.). *Complement to semantic segmentation focussed on instance specific detection using bounding boxes. In essence the question of what?, where?*

Evaluation.

- **Precision:** accuracy of predictions (how many correct?): $\frac{TP}{TP+FP}$
- **Recall:** detection of true objects: $\frac{TP}{TP+FN}$
- **IoU:** overlap of two bounding boxes (i.e., predicted vs ground truth)
- **(m)AP:** The **Average Precision** is the **Area Under the Curve** (Precision-Recall). Core idea is to take ranked detection by confidence level and some IoU threshold (when does a prediction count as correct). Then the **Precision-Recall Curve** is given by these ranked predictions. Since Prec and Rec $\in [0, 1]$ the unit integral of AP is $\in [0, 1]$. We ensure monotonic decrease by replacing each precision value with the maximum precision value to the right of that recall level.
- **Interpolated AP:** use an n -point interpolated AP, and then use the mean-AP of these n points.

AP Note: Generally the AP is determined for a given class and iou-threshold. Generally, we apply a number of predefined thresholds (loose to tight) and average them for a given class. All per class APs are average at the end for the full AP $\rightarrow$ *average (class), average (IoU thresholds), average (precision @ different recall levels) precision.*

2.2.2 Two Stage R-CNN

Region-based CNNs use per-region computation for the detection task, rather than considering the full input in one loss function target. So the classifier only ever sees some number of potentially high quality regions, not the full $H_f W_f$ anchor grid.

Note: One-Stage detectors, have no separate proposal step. Every anchor is directly classified into $K + 1$ classes and regressed in a single forward pass (no proposal, every anchor, 1 box regression, faster, but lower quality).

R-CNN

1. **Region Proposals:** off-the-shelf proposal algorithm (selective search / Edge Boxes / MCG) – *not* a learned network (the RPN is Faster R-CNN’s innovation), and trained disjoint from this architecture.
2. **Crop & Warp** the patches for fixed network input size
3. **Forward** through network for feature map $\rightarrow$ VERY SLOW per-region pass required (≈ 2000)
4. **Linear Object Classifier** plus a parallel **Box Regressor** for a refined proposal localization

Fast R-CNN

1. **FCN transformation** of input into featurized repr. $f_I = f(I)$
2. unchanged off-the-shelf region proposals $r(I) \rightarrow$ still the bottleneck: slow and limited recall (this is the motivation for Faster R-CNN)
3. **RoI Pooling** for a featurized repr of each proposal $g_i = g(f_I, r_i)$: project proposal to feature-map coords (divide by FCN stride s), snap fractional coords to integer feature-map cells, split into fixed $k \times k$ grid, max-pool per bin $\rightarrow$ fixed $k \times k \times C$ output regardless of proposal size.
4. **ConvNet/MLP** for preprocessing per proposal feature (input already has lower spatial resolution since we use feature maps) $\rightarrow h(g_i)$
5. **Multiple Heads** can be attached to these feature maps for task-specific predictions.

(Multi-Task) Loss

Since we have two sibling output layers we need to combine their losses for joint training.

- $L_{cls}(p, u) = -\log p_u$ as discrete probability dist per RoI over $K+1$ categories (CE-loss)
- $L_{loc}(t^u, v) = \sum_{i \in \{x, y, w, h\}} \text{smooth}_{L_1}(t_i^u - v_i)$ summing per-coord robust offsets, with $\text{smooth}_{L_1}(x) = 0.5x^2$ if $|x| < 1$ else $|x| - 0.5$ (Huber: L_2 near 0, L_1 in tails $\rightarrow$ less outlier-sensitive). Predicts $t^k = (t_x^k, t_y^k, t_w^k, t_h^k)$ for each of the K categories.
- **Combined:** each RoI is given a ground-truth class u and a BB-regression target v . The BB-loss is activated for all non-background classes ($\lambda[u \geq 1]$)

$$L(p, u, t^u, v) = L_{cls}(p, u) + \lambda[u \geq 1]L_{loc}(t^u, v)$$

Faster R-CNN

1. unchanged FCN transformation of input into featurized repr. $f_I = f(I)$
2. **Learned Region Proposals** using the already transformed f_I (computation is shared) $\rightarrow RPN(f_I)$:
 A 3×3 *sliding window* scans f_I ; at each position it reads a feature vector that is fed to two sibling heads. At the same position we place k predefined *anchor boxes* (cartesian product of scales $\times$ ratios, e.g. $3 \times 3 = 9$). **Crucially the two boxes are decoupled:** the 3×3 window only defines *where features are read from*; the anchor is the *reference frame for the predictions*. The heads output, *per anchor*:
 - **objectness** $\in [0, 1]$: does *this anchor* (not the sliding window) contain an object?
 - **box regression** (dx, dy, dh, dw) : offsets *relative to the anchor*, not the sliding window.
 So a single feature vector yields k proposals at different scales/ratios sharing one center – the anchor sets the prior, the deltas fine-tune. Per-location output: $2k$ scores + $4k$ regression values $\rightarrow$ up to k co-located objects detectable per sliding position. Total $H_f W_f k$ proposals, filtered by objectness threshold + NMS before being passed as RoIs.
3. unchanged RoI Pooling, unchanged ConvNet/MLP, unchanged Multiple Heads

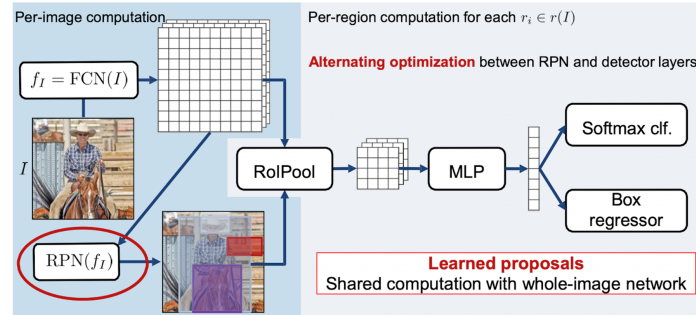


Figure 5: Faster R-CNN: per-image FCN backbone is shared between the RPN (learned proposals) and the per-region head (RoIPool $\rightarrow$ MLP $\rightarrow$ softmax + box regressor). Training alternates between RPN and detector layers.

RPN Loss & Anchor Assignment

Each anchor i gets a label p_i^* :

- **positive** ($p_i^* = 1$): IoU > 0.7 with any GT box (or the highest-IoU anchor for a GT, to guarantee a match)
- **negative** ($p_i^* = 0$): IoU < 0.3 with all GT boxes
- **ignored**: in between – no loss contribution.

One GT can match *multiple* anchors. Multi-task loss combines objectness CE and box-regression smooth- L_1 :

$$\mathcal{L}(\{p_i\}, \{t_i\}) = \frac{1}{N_{\text{cls}}} \sum_i \mathcal{L}_{\text{cls}}(p_i, p_i^*) + \lambda \frac{1}{N_{\text{reg}}} \sum_i p_i^* \mathcal{L}_{\text{reg}}(t_i, t_i^*)$$

$\mathcal{L}_{\text{cls}}$ runs over *all* pos+neg anchors; $\mathcal{L}_{\text{reg}}$ is gated by p_i^* – only positives get a regression loss (negatives have no GT box to regress to). Architecturally the sliding window feeds a 256-d intermediate vector that branches into the $2k$ -d cls head and $4k$ -d reg head.

Faster R-CNN with Feature Pyramid Network (FPN)

Rather than using a same scale feature map, we include a multi-scale feature pyramid, i.e., f_I is no longer just one repr at one scale. This improves *Scale Invariance* and *Localization Equivariance*.

Different Strategies:

- **Image Pyramid**: At each scale predict individually - standard, easy but SLOW
- **Multiscale Features (Single-scale Map)**: single feature map by downsampling - fast but suboptimal especially for small objects
- **Naive In-Network Pyramid**: use intermediate features while downsampling for predictions - fast, suboptimal
- **Feature Pyramid Network**: enrich high-res features from low-res features in a top-down manner - fast, less suboptimal, since we use the strong features of low resolution with the fine grained nature of the high resolution

Issues with Indirect Detectors

Anchor-based methods are *indirect*: predictions are made w.r.t. a hand-designed anchor grid rather than the objects themselves. Two pain points:

- **Heuristic GT→anchor assignment**: IoU thresholds (0.7/0.3), one GT → multiple anchors – hyperparameters baked into training.
- **Duplicate predictions**: k anchors per location × overlapping receptive fields → many near-duplicate boxes. Requires *NMS* post-processing at inference time (non-differentiable, extra hyperparameters).

2.2.3 Direct Detectors

In order to avoid brittle heuristics we can apply direct detectors that drop anchors + NMS entirely.

Idea: A direct detector outputs a fixed-size set of N predictions $\hat{y} = \{\hat{y}_i\}_{i=1}^N$, where N is chosen larger than the max number of objects in any image. No anchors, no grid; just N slots, each producing $(\hat{p}_i, \hat{b}_i) = (\text{class distribution, box})$. The ground-truth set y is padded with "no object" $\emptyset$ entries up to size N so both sides have the same cardinality.

Problem: The network outputs N predicted boxes in arbitrary order. In the indirect methods, we used anchor → closest GT matching. Here we no longer have these heuristics.

Direct Set Prediction

Via **Bipartite Matching** (NOT training), we can solve the matching as a permutation problem, s.t., σ of predictions that pairs each prediction with exactly one GT (or $\emptyset$) so that the total matching cost is minimized:

$$\hat{\sigma} = \arg \min_{\sigma} \sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

$$\text{with } y_i = (c_i, b_i) \quad \hat{y}_{\sigma(i)} = (\hat{p}_{\sigma(i)}, \hat{b}_{\sigma(i)})$$

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) = -[c_i \neq \emptyset] \hat{p}_{\sigma(i)}(c_i) + [c_i \neq \emptyset] \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

combining both the class probability ($\hat{p}_i$) and the box similarity ($\hat{b}_i$) → solved in polynomial time using the **Hungarian Algorithm**. Note that we use the non-empty-set instances for both the class prob and the box loss. We do not care about predictions assigned to $\emptyset$, what we want is ONLY the best matches for the GT. With this we do not punish $\emptyset$ -assignments after having found the best GT-matches.

On the best match, we then calculate the **Set Loss**/ Hungarian Loss, where we must tell the network that an unmatched slot should have predicted $\emptyset$ (training), otherwise it would not learn to predict this:

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=0}^N -\log \hat{p}_{\hat{\sigma}(i)}(c_i) + [c_i \neq \emptyset] \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \quad \text{NLL/CE-loss for strong training signals}$$

Detection Transformer

We can use DETR to generate *direct box predictions*.

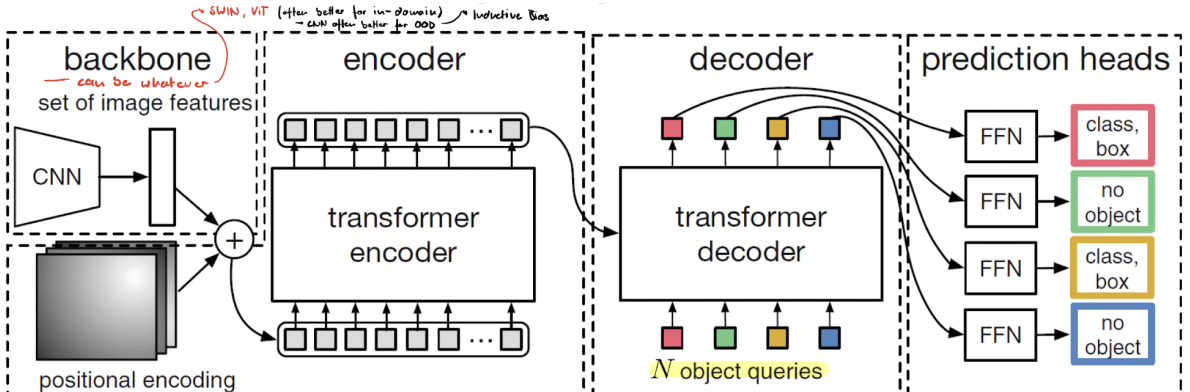


Figure 6: DETR architecture: CNN backbone + positional encoding → transformer encoder over flattened image features → transformer decoder cross-attending N learned *object queries* against the encoded features → shared FFN heads output (class, box) per query, with $\emptyset$ for unused slots.

- **Positional Encoding**: added in the encoder to distinguish same features generated at different spatial locations (transformer is permutation-invariant otherwise).

- **Object Queries:** N learnt input embeddings added to the input of each attention layer in the decoder $\rightarrow$ each query becomes one prediction slot.
- **Decoder Self-Attention:** lets queries attend to each other $\rightarrow$ pair-wise relations between objects (the mechanism that suppresses duplicates: queries learn to "divide up" the image).
- **Encoder-Decoder Attention** (cross-attention): each query attends over the full encoded feature map $\rightarrow$ whole image as context per object.
- **Prediction Heads:** FFNs with *shared parameters* but *independent computation per query* $\rightarrow (c_i, b_i)$ per slot.

2.3 Instance Segmentation

Definition 10 (Instance Segmentation). *Object Detection with pixel-level masks instead of bounding boxes for objects. Evaluation still via AP.*

2.3.1 Mask R-CNN

We still evolve the R-CNN lineage. The idea is now to *detect-then-segment*.

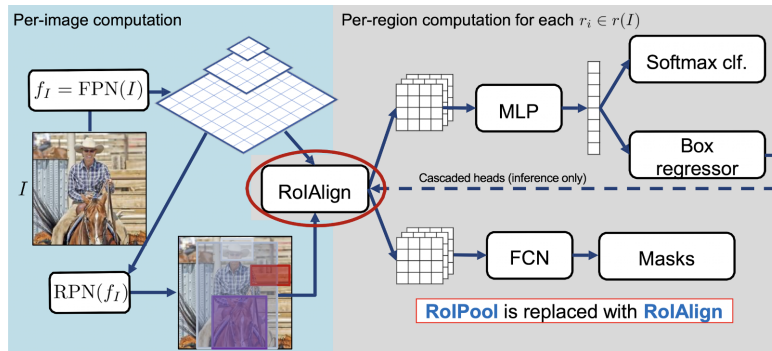


Figure 7: Mask R-CNN: adapted Faster R-CNN with an instance-level segmentation mask head, and RoIAlign.

Core Adaptations of the Mask R-CNN:

- **Instance-Level Segmentation Mask Head:** Decoupled Segmentation from classification such that one mask prediction for each (proposal, class) pair.
- **RoIAlign:** Same concept of transforming an arbitrary-sized proposal into a fixed-dimensional repr, but *without any quantization*.
Why alignment is needed at all: the RPN reads features from integer feature-map cells but writes boxes in continuous image-pixel space (deltas $(dx, dy, dw, dh) \in \mathbb{R}^4$ are continuous on top of the anchor). So when the head re-reads features for that proposal, the box no longer lands on grid cells – that gap is what RoIPool/RoIAlign bridge.

Recall RoIPool snaps twice – (i) project RoI to integer feature-map cells, (ii) snap bin boundaries when subdividing – introducing $\sim$ stride-sized misalignments. Fine for classification, but fatal for per-pixel mask prediction. RoIAlign keeps everything fractional:

1. Project the RoI to the feature map – *do not snap*.
2. Subdivide into a $k \times k$ grid of bins – *do not snap* boundaries.
3. Inside each bin, place a fixed number of *sample points* at exact fractional positions (e.g. uniform $2 \times 2 = 4$ samples per bin).
4. For each sample, get its feature value via **bilinear interpolation** from the 4 nearest feature-map lattice points (features are treated as living at grid *junctions*, not cell centers).
5. Bin output = *average* of that bin's sample points $\rightarrow$ output spatial size = $k \times k$ (the number of *bins*), *independent* of how many sample points each bin averages.

Output is still $k \times k \times C$, but spatially faithful and *differentiable* w.r.t. RoI coordinates (RoIPool is not). Key for masks: $14 \times 14 / 28 \times 28$ mask predictions are later resized to the image-space RoI and thresholded at 0.5, so any RoIPool-style quantization error would propagate directly to mask boundaries.

Mask R-CNN Loss

Builds on the Fast R-CNN multi-task loss by adding a per-pixel mask term. For each proposal the mask head outputs a $K \times m \times m$ feature map f (one $m \times m$ mask per class, no softmax across classes), passed through a sigmoid $\rightarrow$ scores s .

- $L_{\text{mask}}(s^u, w) = -\frac{1}{m^2} \sum_{i=1}^{m^2} [w[i] \log s^u[i] + (1 - w[i]) \log(1 - s^u[i])]$ per-pixel *binary* CE on the sigmoid scores of the mask s^u corresponding to the GT class u , against the binary GT mask w . Decouples class from mask – no inter-class competition per pixel (unlike softmax).
- **Combined:** mask loss only on non-background proposals ($[u \geq 1]$):

$$L(p, u, t^u, v, s^u, w) = L_{\text{cls}}(p, u) + [u \geq 1] L_{\text{box}}(t^u, v) + [u \geq 1] L_{\text{mask}}(s^u, w)$$

2.4 Panoptic Segmentation

2.4.1 Fundamentals and Definitions

Definition 11 (Panoptic Segmentation). *For a given, pre-defined set of classes $\mathcal{C}$, assign each pixel in the input image individually to a single class. For each pixel, that belong to a countable class, assign it to a unique instance.*

Panoptic Segmentation unifies the instance and classical semantic segmentation into one task, using a uniform output format and a uniform evaluation metric (**Panoptic Quality**).

Data is in generic supervised learning form: $\mathcal{D} = \{(x_i, y_i) : i = 1, \dots, N\}$. Thus, with Z as the max number of instances for a given class:

Input (RGB): $x_i \triangleq \mathbf{I}_i \in \mathbb{R}^{H \times W \times 3}$

Output: $\mathbf{Y}_i \in \{1, \dots, C\} \times \{1, \dots, Z\}^{H \times W}$

Loss: depending on the architecture (instance-semantic or unified mask-based)

For each pixel i : (**class, instance count (if countable)**). Contrary to instance segmentation we have **no prediction overlaps** (but arbitrary instance ordering), and **no instance confidence scores**.

Panoptic Quality (PQ).

Fundamental Principle: Each GT segment can have at most one corresponding predicted segment with $\text{IoU} > 0.5$ and vice versa. Thus matching is unique and finding it is simply *the first match with $\text{IoU} > 0.5$* . Matching uses *only* $\text{IoU} > 0.5$ – the raw instance IDs are arbitrary and ignored, so a predicted segment's ID need *not* equal the GT segment's ID for them to match.

$$\text{PQ} = \frac{\sum_{(p,g) \in TP} \text{IoU}(p, g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|} \quad \text{for a single class}$$

We can interpret this as a product of segmentation quality (SQ) and recognition quality (RQ):

$$\text{PQ} = \underbrace{\frac{\sum_{(p,g) \in TP} \text{IoU}(p, g)}{|TP|}}_{\text{SQ}} \times \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{\text{RQ}}$$

- **RQ** = F1 score (harmonic mean of precision & recall) – detection quality.
- **SQ** = average IoU over matched (TP) segments – localization/mask quality.
- SQ depends on RQ (only the matched segments enter SQ).
- Decomposition only holds *per class* – does not hold for mean PQ across classes.

2.4.2 Instance-semantic Framework for Panoptic Segmentation

Idea: re-use existing instance and semantic segmenters to obtain a panoptic output without designing a new model $\rightarrow$ combine their predictions post-hoc.

Baseline: Merge Independent Predictions

Run an instance segmenter (Mask R-CNN, things only) and a semantic segmenter (stuff + things) independently, then merge pixel-wise: things win over stuff; overlapping instances resolved by confidence.

- PQ^{Th} unchanged (instances taken as-is); PQ^{St} drops slightly (instance masks carve into stuff).
- **Failure mode:** *unlabeled (black) halos around boundaries* – semantic seg predicts a thing class but no instance mask covers the pixel $\rightarrow$ no valid instance ID $\rightarrow$ pixel unlabeled. Structural flaw of independent models.
- Motivates *backbone-sharing* models (Panoptic FPN).

Panoptic FPN: Shared Backbone

Motivation: the baseline fails structurally because the two segmenters never share features $\rightarrow$ no way to agree on boundaries (black halos). Sharing a backbone (a) forces the features to be jointly useful for both tasks, (b) is essentially free since FPN already produces *high-resolution, rich, multi-scale* features – exactly the standard requirements for semantic seg.

Starting point: FPN-based Mask R-CNN. Idea: add a *semantic segmentation branch* fed by the same FPN $\rightarrow$ instance + semantic heads jointly trained on a shared feature pyramid.

- **Semantic branch:** each FPN level (resolutions $1/32, 1/16, 1/8, 1/4$ with 256 channels) is upsampled to a common $1/4$ resolution via a stack of *conv* $\rightarrow 2\times$ *upsample* blocks (more blocks for coarser levels) producing $128 \times 1/4$ maps, element-wise summed, then *conv* $\rightarrow 4\times$ to image resolution $\rightarrow$ per-pixel C -way logits.
- **Training loss:** weighted sum to account for the different ranges of the two losses (instance heads are per-RoI, semantic head is per-pixel over the full image): $L = \lambda_{\text{inst}}(L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}) + \lambda_{\text{sem}}L_{\text{sem}}$
- **Panoptic inference** (heuristic merge – panoptic format forbids overlap):
 - instance-instance overlap $\rightarrow$ higher confidence wins;
 - instance-stuff overlap $\rightarrow$ instance wins;
 - stuff segments below an area threshold are dropped.
- **Limitation:** heads are still *segregated* – they share a backbone but make independent predictions reconciled by hand-crafted rules $\rightarrow$ suboptimal post-processing. Motivates the unified mask-based formulation below.

2.4.3 Unified Mask-based Framework for Segmentation

Rather than disjointly learning instance and semantic segmentation, we combine them into one unified objective.

Segmentation as Mask-based Classification

Both instance and semantic seg can be written as predicting a *set* of (*mask*, *class*) pairs. This lets us reuse DETR’s set-prediction recipe, only swapping the geometric output from a *box* to a *mask*.

- **Ground truth:** $z^{\text{gt}} = \{(c_i^{\text{gt}}, m_i^{\text{gt}})\}_{i=1}^{N^{\text{gt}}}$ with $c_i^{\text{gt}} \in \{1, \dots, K\}$, $m_i^{\text{gt}} \in \{0, 1\}^{H \times W}$.
- **Prediction:** $z = \{(p_i, m_i)\}_{i=1}^N$ – N slots, each a class *distribution* p_i and a *soft* mask $m_i \in [0, 1]^{H \times W}$.
- **Matching:** pad z^{gt} with $\emptyset$ tokens up to N , then bipartite-match with the Hungarian algorithm (cf. DETR).

Matching cost per pair (both terms gated on $c_i^{\text{gt}} \neq \emptyset$; raw p – no log – so the two terms stay on the same numerical scale and neither dominates the matching, same logic as DETR):

$$\mathcal{L}_{\text{match}}(z_{\sigma(i)}, z_i^{\text{gt}}) = -[c_i^{\text{gt}} \neq \emptyset] p_{\sigma(i)}(c_i^{\text{gt}}) + [c_i^{\text{gt}} \neq \emptyset] \mathcal{L}_{\text{mask}}(m_{\sigma(i)}, m_i^{\text{gt}})$$

$\mathcal{L}_{\text{mask}}$: binary mask overlap loss (details later).

Hungarian / set loss (a.k.a. *mask-classification loss*) given the optimal match $\hat{\sigma}$:

$$\mathcal{L}_{\text{mask-cls}}(z, z^{\text{gt}}) = \sum_{i=1}^N -\log p_{\hat{\sigma}(i)}(c_i^{\text{gt}}) + [c_i^{\text{gt}} \neq \emptyset] \mathcal{L}_{\text{mask}}(m_{\hat{\sigma}(i)}, m_i^{\text{gt}})$$

Same DETR-style asymmetry: the cls term is *not* gated by $[c_i^{\text{gt}} \neq \emptyset]$ – unmatched slots are explicitly supervised toward the $\emptyset$ class to suppress duplicates; switching to $-\log p$ gives proper NLL gradients during training. Mask term stays gated (no GT mask for $\emptyset$ slots).

MaskFormer

Similar idea as DETR but *without a transformer encoder* – segmentation needs per-pixel features at near-image resolution, and self-attention is $\mathcal{O}(N^2)$ in tokens, so flattening high-res feature maps into an encoder would be prohibitive. Instead a *pixel decoder* (FPN-style CNN upsampling) produces the dense per-pixel features that the transformer decoder’s queries cross-attend against.

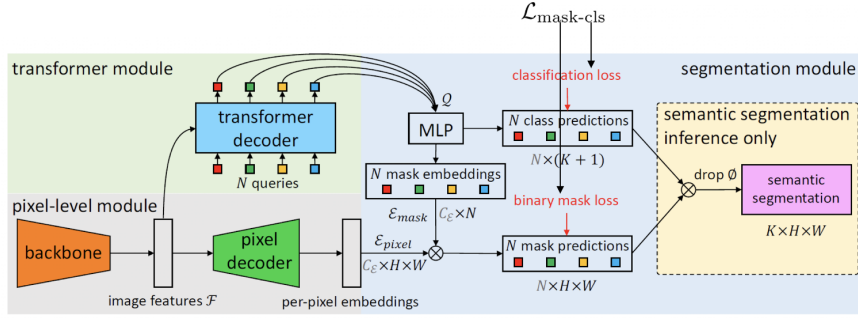


Figure 8: MaskFormer High Level View

Mask Prediction

Sigmoid dot product of per-query *mask embedding* $\mathcal{E}_{\text{mask}}[:, i]$ with per-pixel embedding $\mathcal{E}_{\text{pixel}}[:, h, w]$ from the pixel decoder: $m_i[h, w] = \sigma(\mathcal{E}_{\text{mask}}[:, i]^T \cdot \mathcal{E}_{\text{pixel}}[:, h, w])$ Interpreted as the *non-exclusive degree* to which pixel (h, w) belongs to mask i – no softmax across slots, so different slots’ masks can overlap. The panoptic no-overlap constraint is enforced only at inference (cf. Mask R-CNN’s per-class masks, here per-query).

Same Architecture, Different Weights

One architecture for semantic / instance / panoptic, but a *separate model is trained per task* – unified *formulation*, not unified *weights* (Mask2Former goes further).

Inference

Training is fixed (mask-cls loss); only the inference rule switches per task:

- **Panoptic / instance:** per pixel, pick the *dominant* (slot, class) pair, $\arg \max_{i: c_i \neq \emptyset} p_i(c_i) \cdot m_i[h, w]$ – both class- and mask-conf must be high. Each instance stays a distinct segment.
- **Semantic:** *marginalize* over slots, $\arg \max_{c \in \{1, \dots, K\}} \sum_{i=1}^N p_i(c) \cdot m_i[h, w]$. Semantic only cares about the class per pixel, not which slot produced it $\rightarrow$ multiple ”person” slots can pool their evidence into one ”person” region.
- Training the semantic objective directly would scatter the gradient across all N queries (each gets only a fraction of the signal) $\rightarrow$ keep mask-cls loss, switch inference only.
- However, specialized methods require significantly fewer epochs for equivalent performance

Mask2Former

Same fundamental idea as MaskFormer but with significant improvements:

- **Masked Attention** (in cross-attention): restrict each query to attend *only to pixels inside its current mask prediction*, by adding an attention bias $\mathcal{M}_{l-1}(x, y) = \begin{cases} 0 & \text{if } M_{l-1}(x, y) = 1 \\ -\infty & \text{otherwise} \end{cases}$ inside the softmax: $X_l = \text{softmax}(\mathcal{M}_{l-1} + Q_l K_l^T) V_l + X_{l-1}$. $-\infty$ entries vanish after softmax $\rightarrow$ those pixels contribute zero. Intuition: *global context takes time to learn*; for cross-attention local foreground features already suffice, broader context is gathered by self-attention.
- **Swap Self- and Cross-Attention:** queries at the first decoder layer are image-independent (purely learned embeddings) $\rightarrow$ self-attention among them carries no image content to share. Do cross-attention first to inject image features into queries, *then* self-attention – opposite of the transformer default.
- **High-Res Features from Pixel Decoder:** MaskFormer decoded queries against the *low-res bottleneck* features only; Mask2Former feeds each successive transformer-decoder layer a *different scale* from the pixel decoder (multi-scale). Helps segment objects of diverse scales, especially small ones.
- **Point Sampling** (matching cost & mask-cls loss approximation): $\mathcal{L}_{\text{mask}}$ over the full $H \times W$ map for every (prediction, GT) pair is memory-heavy. Approximate it on K sampled pixels:

- *Matching cost*: *uniform* sampling of a *shared* pixel set across all pairs $\rightarrow$ costs stay comparable across pairs (apples-to-apples for the Hungarian).
- *Mask-cls loss*: *importance* sampling of *distinct* pixel sets per mask $\rightarrow$ each mask gets its own boundary/hard-pixel focus.

Drops the full-image mask compute requirement $\rightarrow$ enables training on standard hardware.

3 Part III: Geometric Perception and Localization

3.1 Depth and 3D Estimation

Stereo vision to generate 3D renderings from 2D images. Other possibilities include: *structured light*, *ultrasound*, *radar*, *time of flight (active)*, and *cam/lidar combinations*. We can deduce some information also from 2D images using cues including shading, texture, focus, and motion.

Challenges to monocular depth estimation: a single 2D image may be produced from an infinite number of 3D scenes due to **scale ambiguity** → overcome some of this by using cues or features such as perspective or texture information, object sizes, locations and occlusions.

→ regression problem (cont output per pixel), thus we use MSE or variants. (no longer MAP!)

Ground Truth.

- **LiDAR** (e.g. Velodyne): sparse; limited vertical FoV; errors on moving objects and on specular/transparent surfaces. In practice can be *densified* for static parts by merging consecutive frames (imperfect, but less sparse).
- **Structured light** (e.g. Kinect): fails outdoors (sunlight interference); short range (~few meters); errors on specular/transparent surfaces.

Evaluation Metrics

Comparing predicted depth map Y to GT Y^* over T valid depth pixels.

- **Threshold accuracy**: fraction of pixels with $\max\left(\frac{y_i}{y_i^*}, \frac{y_i^*}{y_i}\right) < \text{thr}$ (symmetric ratio ≥ 1 → penalizes over- and under-estimation). Counts correct pixel given some tolerance; typical threshold 1.25. Robustness-flavored.
- **Absolute relative difference**: $\text{rel} = \frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*| / y_{i,j}^*$ – per-pixel error normalized by GT depth (far pixels not over-weighted).
- **Squared relative difference**: $\text{srel} = \frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*|^2 / y_{i,j}^*$ – penalizes larger errors more.
- **RMS (linear)**: $\sqrt{\frac{1}{T} \sum_{i,j} |y_{i,j} - y_{i,j}^*|^2}$ – standard L2; dominated by far/large-depth errors.
- **RMS (log)**: $\sqrt{\frac{1}{T} \sum_{i,j} |\log y_{i,j} - \log y_{i,j}^*|^2}$ – compares depth *magnitudes* (orders of magnitude); equalizes near/far contributions.

3.1.1 Supervised Methods

Standard CNN based Enc-Dec methods with loss between output and GT depth.

Loss: While $L1$ and $L2$ are possible, the **berHu loss** is more common: $\mathcal{B}(x) = \begin{cases} |x| & |x| \leq c \\ \frac{x^2 + c^2}{2c} & |x| > c \end{cases}$

Inverse of Huber: $L1$ for small residuals, $L2$ for large ones.

Computed as $\mathcal{B}(\tilde{y} - y)$ with $c = \frac{1}{5} \max_i (|\tilde{y}_i - y_i|)$ over all pixels in the current batch – the $L1/L2$ cut-off is set to 20% of the largest residual in the batch (adaptive scale).

P3Depth.

Leveraging regularities in real-world (planar) scenes yielding piecewise-smooth depth maps. → rather than directly predicting depth D , predict a per-pixel repr $\mathbf{C}$ that is *identical for pixels on the same 3D plane*, then use neighbor information in $\mathbf{C}$ to predict the depth of each pixel.

Plane Coefficient Representation

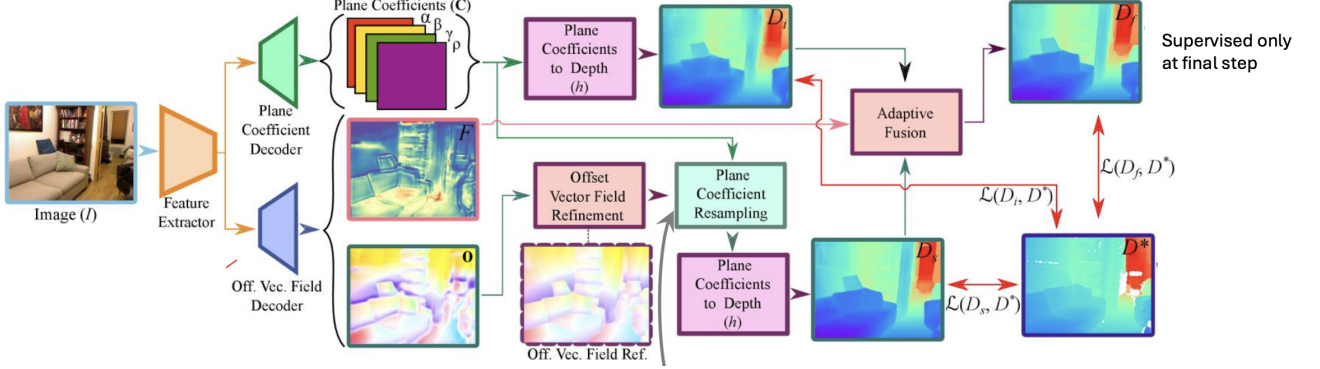
Backproject a pixel $\mathbf{p} = (u, v)^\top$ to a 3D point $\mathbf{P} = (X, Y, Z)^\top$ via the pinhole model: $Z = D(u, v)$, $X = \frac{Z(u - u_0)}{f_x}$, $Y = \frac{Z(v - v_0)}{f_y}$. A 3D plane with normal $\mathbf{n} = (a, b, c)^\top$ and offset d satisfies $\mathbf{n} \cdot \mathbf{P} + d = 0$. Substituting the backprojection and rearranging gives a *linear function for $1/Z$ in pixel coords*:

$$\frac{1}{Z} = \underbrace{\frac{-a}{f_x d}}_{\hat{\alpha}} u + \underbrace{\frac{-b}{f_y d}}_{\hat{\beta}} v + \underbrace{\frac{1}{d} \left(\frac{a}{f_x} u_0 + \frac{b}{f_y} v_0 - c \right)}_{\hat{\gamma}} \quad \text{Normalize: } \rho = \sqrt{\hat{\alpha}^2 + \hat{\beta}^2 + \hat{\gamma}^2} \quad (\alpha, \beta, \gamma) = (\hat{\alpha}, \hat{\beta}, \hat{\gamma}) / \rho$$

→ **plane coefficients** $\mathbf{C} = (\alpha, \beta, \gamma, \rho)^\top$. Depth recovered from $\mathbf{C}$ and pixel coords: $Z = [(\alpha u + \beta v + \gamma)\rho]^{-1} = h(\mathbf{C}, u, v)$. Pixels on the same 3D plane share the same $\mathbf{C}$ – it depends only on camera intrinsics and the plane, not the individual pixel → enables neighborhood-based depth refinement.

Injecting Regularities

In order to actually use these **interaction between pixels in the new output space**, we must fuse the spaces. Simply changing the output space to a repr more closely related to 3D planes does NOT inject these priors.



Two decoders branch off a shared feature extractor:

- **Plane Coefficient Decoder** → dense plane-coefficient map $\mathbf{C}(u, v) = (\alpha, \beta, \gamma, \rho)$.
- **Offset Vector Field Decoder** → per-pixel offset $\mathbf{o}(\mathbf{p}) = \mathbf{q} - \mathbf{p}$ pointing to a **seed pixel** $\mathbf{q}$ in the neighborhood that is associated with the *same 3D plane* as $\mathbf{p}$.

Two depth predictions are then computed:

- **Initial:** $D_i(u, v) = h(\mathbf{C}(u, v), u, v)$ – each pixel uses its own coefficients.
- **Seed-based:** resample coefficients at the seed location, $\mathbf{C}_s(\mathbf{p}) = \mathbf{C}(\mathbf{p} + \mathbf{o}(\mathbf{p}))$, then $D_s(u, v) = h(\mathbf{C}_s(u, v), u, v)$ – uses the seed’s plane params but $\mathbf{p}$ ’s pixel coords (valid since both share the same plane).

Why the seed at all? CNN/transformer decoders produce smooth feature maps → plane coefficients are reliable in plane *interiors* but blur near plane boundaries. The offset snaps each pixel to a more reliable interior seed of its own plane → sharp depth discontinuities at edges.

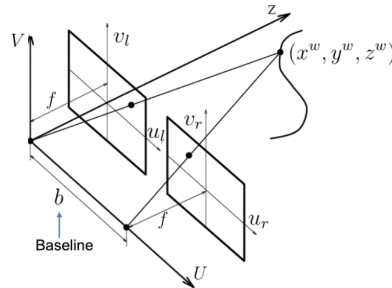
Adaptive Fusion: $D_f(u, v) = F(u, v)D_s(u, v) + (1 - F(u, v))D_i(u, v)$, with predicted confidence $F \in [0, 1]$. High F trusts the piecewise-planar prior; low F falls back to direct prediction for non-planar surfaces (curved, high-texture).

Training: supervised *only on depth*: $\mathcal{L}(D_i, D^*) + \mathcal{L}(D_s, D^*) + \mathcal{L}(D_f, D^*)$. Offsets, plane coefficients, and confidence are all learned *implicitly* – no GT for them.

3.1.2 Stereo Depth Estimation

Using a stereo setup we can estimate depth using triangulation.

We consider *identical cameras, with coplanar image planes, aligned x-axes and the convention of setting the left camera frame to the world coords*. Thus have a translation along the known baseline (in meters) and no rotation.



The translation from the $C_L = W$ to C_R is then given by:

$$\begin{pmatrix} x^r \\ y^r \\ z^r \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix} \begin{pmatrix} x^w \\ y^w \\ z^w \\ 1 \end{pmatrix} \quad \text{with} \quad \mathbf{R} = \mathbf{I} \text{ and } \mathbf{C} = (b \ 0 \ 0)^T = -\mathbf{t}$$

Projection & Disparity

Further assume image coords already centered at the principal point $((u_0, v_0) = (0, 0))$ and $K_L = K_R = K$ with $K = \text{diag}(kf, lf, 1)$. Projecting the world point through each camera: $\rho \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x^w \\ y^w \\ z^w \end{pmatrix}$, $\rho' \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = K \begin{pmatrix} x^w - b \\ y^w \\ z^w \end{pmatrix}$

Dehomogenizing:

$$u = \frac{kfx^w}{z^w}, v = \frac{lfy^w}{z^w}, \quad u' = \frac{kf(x^w - b)}{z^w}, v' = \frac{lfy^w}{z^w}$$

$\rightarrow v = v'$: rectified pair, the same scene point lies on the same image row in both cameras (only horizontal disparity).

Disparity: $d = u - u'$ – the horizontal displacement of the same point between the two images. Solving the equations above for the 3D coords:

$$x^w = b \frac{u}{u - u'}, \quad y^w = b \frac{k}{l} \frac{v}{u - u'}, \quad z^w = bk \frac{f}{u - u'}$$

Depth is inversely proportional to disparity. Normally $k = l$ (square pixels). Near objects $\rightarrow$ large d (well-resolved); far objects $\rightarrow$ small d (precision degrades quadratically with distance).

Visible Region

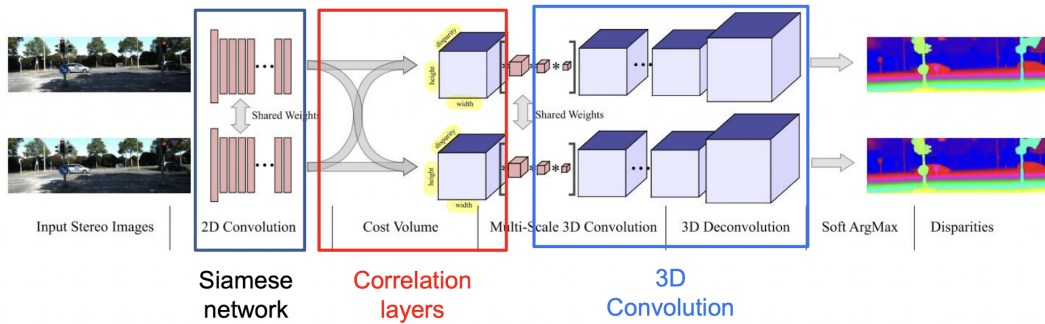
- **Blind region:** zone in front of the cameras visible to only one (or neither) $\rightarrow$ no stereo depth possible.
- **Valid stereo region:** intersection of both FoVs $\rightarrow$ correspondences (hence depth) computable.

Design Trade-offs & Remarks

- Increasing b and/or $f \rightarrow$ better depth resolution, but smaller simultaneous visibility (larger blind region; narrower FoV for f).
- *Same disparity = same depth* holds *only for this rectified simple setup* – not for general stereo geometries.
- Too-small disparity $\rightarrow$ depth becomes unresolvable: sub-pixel matching noise translates to large depth ambiguities (quadratic in distance) $\rightarrow$ far objects unreliable.
- The real bottleneck is **finding correspondences** under non-ideal inputs: occlusions, repetitive patterns, textureless regions, reflective surfaces.

Stereo Matching Algorithm and Siamese Networks.

Classical four-step process: (I) matching cost, (II) cost aggregation, (III) optimization, (IV) disparity refinement. **Siamese Networks** = two twin networks with *shared parameters* processing two inputs in parallel, trained to minimize the distance for matching pairs and maximize it for distinct ones $\rightarrow$ natural fit for step (I).



End-to-end deep stereo regression replaces all four steps with learned blocks:

- **Siamese 2D conv** – shared-weight feature extractor on both images $\rightarrow$ learned per-pixel descriptors (replaces hand-crafted matching cost of step I).
- **Cost volume** (a.k.a. *correlation layer*) – *no learnable parameters*; a fixed construction that for every (u, v) and candidate disparity d combines $f_L(u, v)$ with $f_R(u - d, v)$ – by concatenation $(H \times W \times (D_{\max} + 1) \times 2C)$ or dot product $(H \times W \times (D_{\max} + 1))$. Built in *both directions* (L $\rightarrow$ R and R $\rightarrow$ L) $\rightarrow$ two cost volumes, one anchored per image, enabling downstream left-right consistency / occlusion reasoning. Still differentiable $\rightarrow$ gradients flow back into the Siamese features. It is the *inductive bias* of the architecture (only same-row, fixed-shift pairs are considered).

- **Multi-scale 3D conv / 3D deconv** – propagates context across spatial *and* disparity dimensions → cost aggregation + optimization (steps II+III). *Why 3D conv, not 2D?* 2D conv would slice across (H, W) per disparity independently, decimating the disparity axis; 3D conv treats D as a proper spatial dim and preserves it. So the cost volume / 3D-conv tensor has **three spatial dimensions** ($H \times W \times \text{disparity}$) plus channels – *not* two spatial dims with disparity playing the role of the channel axis. Geometry-motivated design, not a black box. Output: a refined cost volume reduced to a *single feature channel* – $H \times W \times (D_{\max} + 1)$ scalar costs.
- **Soft argmax / soft argmin** – differentiable disparity selection (step III). A hard argmin over costs would (i) only allow integer disparities → quantized depth and (ii) make the loss *fragile*: a prediction one off from GT incurs the same loss as one completely off (no gradient pointing toward correct). Instead treat costs as negated logits, softmax over the disparity axis, take the convex combination: $\hat{d}[i, j] = \sum_{D=0}^{D_{\max}} D \cdot \text{softmax}_D(-C[i, j, D])$ → continuous, sub-pixel disparity that flows gradients smoothly. Final loss: L_1 regression

$$L = \frac{1}{HW} \sum_{i,j} \|\hat{d}[i, j] - d[i, j]\|_1$$

Why a cost volume rather than a naive concat? A naive baseline would just concatenate the left and right feature maps along channels and regress disparity with a big CNN – but this throws away the stereo geometry, forcing the network to *discover* that matches lie along the same row at a small horizontal shift. The cost volume bakes that prior in: each disparity slice d holds the similarity between the left feature at (u, v) and the right feature at $(u - d, v)$ → only same-row, fixed-shift correspondences are considered, so the network just has to *pick* the best disparity.

3.1.3 Self-Supervised Monocular Depth Estimation from Stereo

Key asymmetry: stereo is used at *training* time as a supervision signal; the CNN only ever sees the *left* image.

- **Forward pass:** $\text{CNN}(L) \rightarrow \text{disparity map}$.
- **Loss:** warp R using the predicted disparity (via the stereo projection equations $u = \frac{kf x^w}{z^w}, u' = \frac{kf(x^w - b)}{z^w}$) → synthesized $\tilde{L}$; photometric loss $\|\tilde{L} - L\|$. Right image is supervision only, not input.
- **Why "monocular":** at test time only one camera is needed. Stereo provides cheap, scalable self-supervision
- **Accuracy:** not as good as fully supervised, but decent.
- **Issues:** occlusions, FoV mismatch between cameras (some pixels in L have no valid match in R).

3.1.4 Self-Supervised Depth and Motion Estimation from Video

Extends stereo self-supervision to *monocular video*: no known baseline, so the camera motion between consecutive frames must also be predicted. Train on raw video, deploy on a single image.

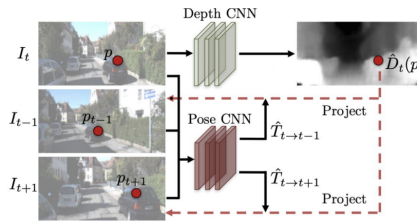


Figure 9: Unsupervised Learning of Depth and Ego-Motion from Video

Two jointly trained networks:

- **Depth CNN:** $I_t \rightarrow \hat{D}_t(p)$ – per-pixel depth of the target frame.
- **Pose CNN:** $(I_t, I_{t\pm 1}) \rightarrow \hat{T}_{t \rightarrow s}$ – relative camera pose (*ego-motion*) between target and source frames.

Warping equation: for a pixel p_t in the target, the corresponding source pixel is $p_s \sim K \hat{T}_{t \rightarrow s} \hat{D}_t(p_t) K^{-1} p_t$ (unproject p_t via K^{-1} , scale by depth, transform by pose, reproject via K). The source image is then sampled at p_s via *bilinear interpolation* (differentiable w.r.t. warp → gradients reach both networks).

View-synthesis photoconsistency loss: $\mathcal{L}_{vs} = \sum_s \sum_p |I_t(p) - \hat{I}_s(p)|$ – L1 between target and warped source frame(s). Depth and pose are supervised *only* through this signal.

Failure modes: breaks on specular / non-Lambertian surfaces (photometric assumption violated); moving objects violate the static-scene assumption → commonly worked around by global motion params or learned masks.

3.2 3D Classification, Segmentation, and Detection

3.2.1 Foundations and Definitions

Representations: many possibilities including multi-view, volumetric (voxels), part assembly, **Point Clouds**, mesh (graph CNN), and implicit shapes (functional repr.) → focus on point clouds since close to raw sensor data and common standard. Point Clouds can either be converted into other reprs to feed into NNs or we can do *feature learning* directly on point clouds (our focus)

3D Perception Tasks: Classification, Segmentation, Detection, Correspondences, Reconstruction

3.2.2 3D Classification and Semantic Segmentation on Point Clouds

PointNet.

Point Cloud based end-to-end learning for *scattered, unordered, sparse, noisy* point data for various tasks. Challenges caused by the data foundation:

- **Unordered** input point set: Model requires invariance to $N!$ permutations (N orderless points). This can be achieved via *Symmetric Functions* (c.f. GNNs): Trivial Solution is the *MLP with max pooling (across points)*, i.e. the max is taken over the N points *per feature dimension* – NOT across features for each point; this is what yields permutation invariance): $f(x_1, \dots, x_n) = \gamma \circ g(h(x_1), \dots, h(x_n))$ symmetric if g is symmetric:

$$\underbrace{x}_{\text{input}} \rightarrow \underbrace{\text{MLP}(x_i)}_h \rightarrow \underbrace{\text{max_pool}}_g \rightarrow \underbrace{\text{MLP}}_\gamma$$
the symmetric g (max-pool) is what ensures permutation invariance; γ just maps the aggregated feature to the output
- **Geometric Transformation Invariance:** rotations / Euclidean transformations must not alter results. Solution: **T-Net** – a mini vanilla PointNet whose output is a transformation *matrix* (not a class score), applied by matrix-mult to align inputs to a *canonical space*. Used twice: 3×3 on raw $n \times 3$ points (*input* T-Net), 64×64 on intermediate $n \times 64$ features (*feature* T-Net). Data-dependent → no hard-coded rotations.

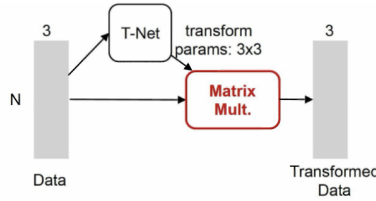


Figure 10: T-Net for Input Alignment by Transformer Network

PointNet Classification Network

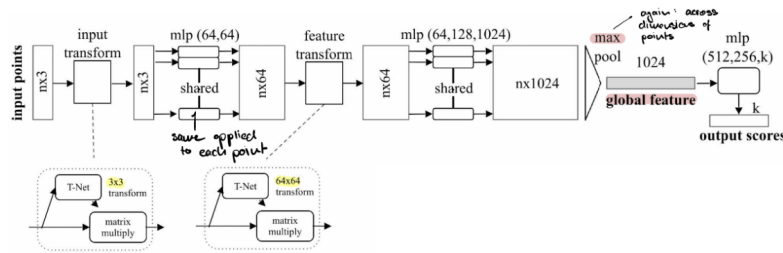


Figure 11: PointNet Classification Network

Whole-cloud → one class. Input: unordered $n \times 3$ points; output: k class scores. Invariances required: permutation (handled by per-point shared MLPs = $h + \text{max-pool} = g$) and rigid transformation (two T-Nets); final MLP = γ .

PointNet Segmentation Network

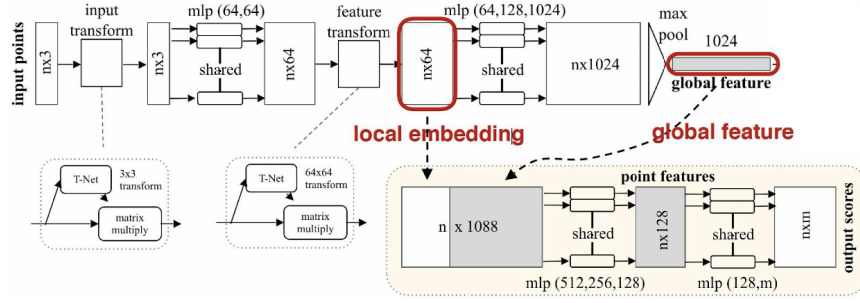


Figure 12: PointNet Segmentation Network

The local embeddings encode point specific details and are non-hierarchical. To each such local embedding we add (concat) the global features so each point sees scene-level context.

PointNet Limitations

- **No local context per point:** features are either *per-point* (one point seen at a time) or *global* (all points aggregated by max-pool) – nothing in between. Contrast with 3D CNNs that have hierarchical feature learning across multiple receptive-field scales.
- **Global feature depends on absolute coordinates:** MLPs ingest raw (x, y, z) , so the global descriptor is tied to absolute position $\rightarrow$ same object at a different location yields a different feature $\rightarrow$ hard to generalize to unseen scene configurations.

PointNet++

Addresses local context per point and global feature dependence on absolute coordinates via hierarchical local-neighborhood processing. The goal is to *mimic convolution* style neighborhood processing.

In scattered point clouds, however, we cannot apply simple convolution, thus we use one of the following to find neighbors:

- **Ball Query:** fix a radius r , take all points within. Fixed metric "kernel" $\rightarrow$ more stable across density variations (the receptive field is the same scale everywhere). Variable number of neighbors.
- **K-NN Query:** distance problematic, since K can represent very distant context if static.

Set Abstraction Levels

Hierarchical structure = a stack of *set abstraction* levels, each repeating three steps:

1. **Sampling Layer:** *Iterative Farthest Point Sampling* (FPS) – given the current sampled set S , the next point is $x_{\text{next}} = \arg \max_{x \notin S} \min_{s \in S} \|x - s\|$ (the unsampled point whose *nearest sampled neighbor* is farthest) $\rightarrow$ each new point fills the largest gap $\rightarrow$ *better coverage* than random sampling.
2. **Grouping Layer:** For each centroid, run ball query (or K-NN) $\rightarrow$ output groups $N' \times K \times (d + C)$. K varies per group (ball query is radius-based); the next PointNet collapses the variable- K axis to a fixed-length feature via max-pool.
3. **PointNet Layer:** Apply a PointNet (*weights shared across all groups of this abstraction level*, like a conv kernel reused across positions) inside each group, taking points *relative* to the centroid $\rightarrow$ one feature per centroid (the analog of one convolution output).

Stacking: N points $(x, y) \rightarrow N_1$ anchors $(x, y, \mathbf{f}) \rightarrow N_2$ anchors $(x, y, \mathbf{f}')$ $\rightarrow \dots$ – fewer points, richer features per level (CNN-style downsampling). Two limitations of vanilla PointNet are fixed: (i) each anchor's feature now *sees its local neighborhood* $\rightarrow$ hierarchical local context; (ii) since inner PointNets see *relative* coordinates, the features are translation-invariant.

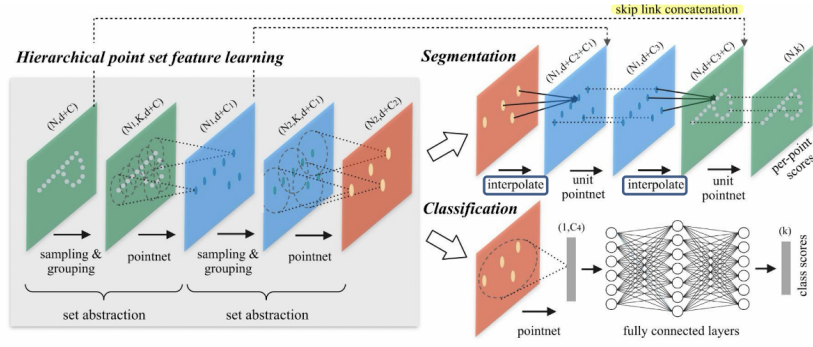


Figure 13: PointNet++ Architecture

Feature Interpolation for PointNet++ Decoder: For per-point tasks (segmentation) we need to undo the set-abstraction subsampling and propagate features back to denser resolution. For each output point $\mathbf{x}_j$, find its k nearest neighbors $\{\mathbf{x}_i\}$ in the (subsampling) input set, then compute an inverse-distance-weighted average:

$$w_{ij} = \frac{1}{d(\mathbf{x}_j, \mathbf{x}_i)^p}, \quad \mathbf{f}(\mathbf{x}_j) = \frac{\sum_{i=1}^k w_{ij} \mathbf{f}(\mathbf{x}_i)}{\sum_{i=1}^k w_{ij}}$$

Closer subsampled points contribute more; typically $k = 3$, $p = 2$. Density-aware upsampling – the decoder analog of bilinear upsampling in 2D CNNs.

3.2.3 3D Object Detection on Point Clouds

Target: 3D BB around each object given 3 DOF for size, 3 DOF for center, and 1 DOF for rotation - assuming object on ground (no vertical rotation).

LiDAR provides reliable depth information used to localize and characterize objects. However, LiDAR point clouds are *sparse, irregular and have highly variable point density*. We can address these challenges by *manually crafting feature repr* for point clouds, *point cloud projection into perspective view* for image-based techniques, or *rasterize point clouds into 3D voxel grids* and encode each voxel with hand-crafted or PointNet features.

VoxelNet

While a pure 2D inspired sliding window and classification approach is theoretically possible, the search space is huge (cubic increase with resolution).

→ two-stage approach **object proposal** + **refinement**

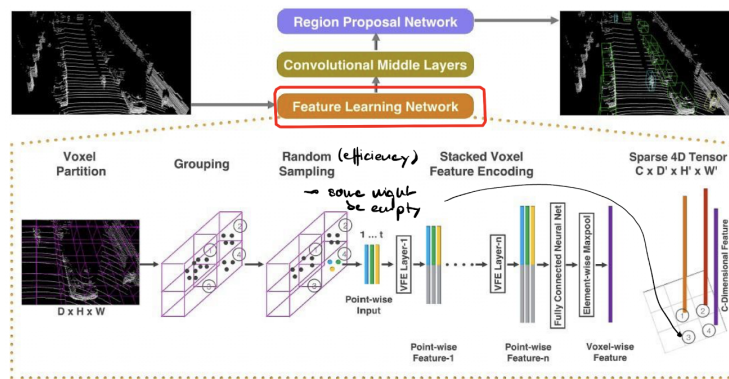


Figure 14: VoxelNet Architecture with Feature Learning Network Details

- **Feature Learning Network:** turns a raw $\sim 100\text{K}$ -point LiDAR cloud into a dense voxel-wise feature grid.
 1. **Voxel Partition:** evenly partition 3D space into a $D \times H \times W$ voxel grid.
 2. **Grouping:** assign each point to its voxel; densities are highly uneven.
 3. **Random Sampling:** cap each voxel at T points → (i) compute savings, (ii) reduces sampling bias from per-voxel point-count imbalance, adds training variation.

4. **Stacked Voxel Feature Encoding (VFE)** – per non-empty voxel $V = \{\mathbf{p}_i = [x_i, y_i, z_i, r_i]^\top\}_{i=1..t}$ (r_i = LiDAR return intensity):
 - Augment each point with its offset to voxel centroid: $\hat{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - v_x, y_i - v_y, z_i - v_z]^\top \in \mathbb{R}^7$.
 - Shared MLP (linear + BN + ReLU) $\rightarrow$ per-point features $\rightarrow$ max-pool across the voxel for a voxel-level feature $\rightarrow$ concat back per point (PointNet-seg local+global).
 - Stack VFE layers; final = one feature per non-empty voxel ($\rightarrow$ sparse 4D tensor).
- **Convolutional Middle Layers:** stack of $3D \text{ conv} + BN + ReLU$ blocks on the sparse voxel tensor. Progressively expanding receptive field aggregates voxel-wise features into richer shape descriptions.
Key step: collapse the vertical (z) dimension to a singleton $\rightarrow$ output is a 2D **BEV (Bird's-Eye-View) feature map**. Driving is essentially planar, so dropping height makes downstream 2D detection feasible.
- **Region Proposal Network:** standard 2D Faster-R-CNN-style RPN, now applicable because the middle layers already produced a BEV map. Two specializations for driving: (i) anchors only at *two* orientations (longitudinal + frontoparallel) since vehicles in traffic appear in essentially two poses, (ii) regression outputs **7-DoF 3D boxes** $(x, y, z, w, l, h, \theta)$ instead of 4-DoF 2D ones – 2D detection geometry, 3D output.

Bounding Box Regression

GT box parameterized as $(x_c^g, y_c^g, z_c^g, l^g, w^g, h^g, \theta^g)$ (center, length/width/height, yaw around z); anchor analogously with superscript a . Residual target $\mathbf{u}^* \in \mathbb{R}^7$ encoding the GT box *relative* to the anchor:

$$\Delta x = \frac{x_c^g - x_c^a}{d^a}, \quad \Delta y = \frac{y_c^g - y_c^a}{d^a}, \quad \Delta z = \frac{z_c^g - z_c^a}{h^a}, \quad \Delta l = \log \frac{l^g}{l^a}, \quad \Delta w = \log \frac{w^g}{w^a}, \quad \Delta h = \log \frac{h^g}{h^a}, \quad \Delta \theta = \theta^g - \theta^a$$

with the anchor's BEV diagonal $d^a = \sqrt{(l^a)^2 + (w^a)^2}$. Normalizing BEV centers by d^a (and height offset by h^a) makes the regression *scale-invariant* in anchor size; *log-ratios* for dimensions turn multiplicative size errors into additive ones and guarantee positivity after exp. Direct analog of Faster R-CNN's 2D box parameterization, extended to 3D + rotation.

Loss

Multi-task loss, Faster-R-CNN style:

$$L = \alpha \frac{1}{N_{\text{pos}}} \sum_i L_{\text{cls}}(p_i^{\text{pos}}, 1) + \beta \frac{1}{N_{\text{neg}}} \sum_j L_{\text{cls}}(p_j^{\text{neg}}, 0) + \frac{1}{N_{\text{pos}}} \sum_i L_{\text{reg}}(\mathbf{u}_i, \mathbf{u}_i^*)$$

- **Classification:** cross-entropy on both *positive* and *negative* anchors (separately normalized), weighted by α, β to balance their relative contribution.
- **Regression:** smooth- L_1 on the 7-D residual $\mathbf{u}_i$, *gated to positive anchors only* – negatives have no GT box to regress against (same gating as Faster R-CNN RPN).

Note: Detection range and per-voxel cap T are tuned *per class*: cars use a wider range (~ 70 m) with $T = 35$; pedestrians/cyclists a shorter range (~ 48 m) with $T = 45$ – smaller targets are denser per voxel but only relevant nearby.

Why operate in 3D? Native 3D operations factor out pose variations and let the model directly exploit geometric structure (planarity, symmetry, repetition) that 2D projections obscure – e.g. the VFE's centroid-relative offsets are a 3D-native prior.

PointRCNN

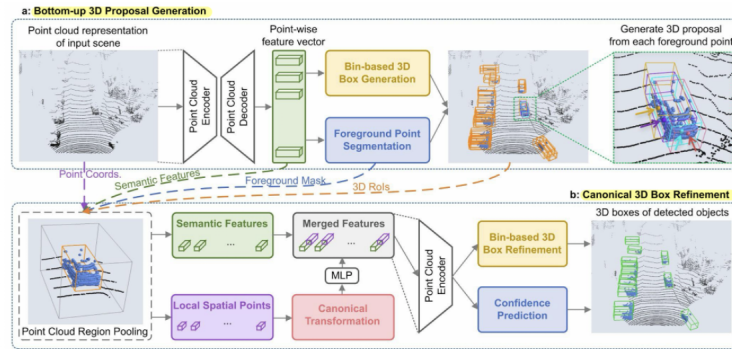


Figure 15: PointRCNN Architecture

- **Bin-based 3D Box Generation:** per foreground point p , instead of regressing the box center as one continuous offset, *discretize the location space into bins* and use a hybrid **classification + residual regression** head.

- Around p , a square location space of half-side $\mathcal{S}$ is split into $\mathcal{C}$ bins per dimension (bin width $\delta = 2\mathcal{S}/\mathcal{C}$). For $u \in \{x, y\}$: $\text{bin}_u^{(p)} = \lfloor (u^p - u^{(p)} + \mathcal{S})/\delta \rfloor$ (GT-center bin index), with residual $\text{res}_u^{(p)}$ measuring the offset within the bin center. Same for yaw θ .
- Other dimensions (z, h, w, l) are regressed directly (no binning – height range is small, sizes are smooth).
- **Hybrid loss**: cross-entropy on bin indices (coarse localization) + smooth- L_1 on residuals (fine offset), averaged over positive proposals. *No objectness loss*: proposals come from foreground points, so they’re always positive by construction.
- *Why*: classification is more *robust for coarse localization* than direct regression – splitting into coarse-class + fine-residual stabilizes training, especially for far/sparse points.
- **Canonical Transformation** (stage-2 refinement): for each proposal, transform its pooled points into a *box-local* Canonical Coordinate System (CCS) – origin at the proposal center, axes aligned with its orientation. Each proposal gets its own CCS. This frees the refinement layers from having to abstract away the proposal’s **3 translational DOF** (its center) and its **rotational DOF around the vertical axis** (its yaw).
 - *Why*: removes the proposal’s global pose dependency → all boxes look alike in their own frame → refinement network learns local geometry consistently across proposals.
 - *What is transformed vs kept global*: only the raw spatial coordinates of pooled points are canonicalized; the *semantic/high-level features* from stage 1 stay in the original frame (they encode global scene context).
 - Canonical spatial features + global semantic features are then *merged* (concat → MLP, then PointNet++ encoder), feeding two refinement heads: a second bin-based 3D box head + a confidence prediction.

3.3 3D Reconstruction and Localization

Definition 12 (Localization in CV.). *CV is the inverse algorithmic process of inferring properties of the world and of optical sensors based on visual sensory observations. The localization then estimates the 3D Euclidean motion as a composite of translation t and rotation R .*

Transformation conventions.

The forward (world/frame 1 → sensor/frame 2) and inverse maps are

$$\mathbf{P}_s = [\mathbf{R} \mid \mathbf{t}] \tilde{\mathbf{P}}_w \Leftrightarrow \mathbf{P}_s = \mathbf{R}\mathbf{P}_w + \mathbf{t} \Leftrightarrow \mathbf{P}_w = \mathbf{R}^T(\mathbf{P}_s - \mathbf{t}) \Leftrightarrow \mathbf{P}_w = \mathbf{R}^T\mathbf{P}_s - \mathbf{R}^T\mathbf{t}.$$

The translation vector is always *the origin of the input frame expressed in coordinates of the output frame*. Hence: $\mathbf{R}, \mathbf{t}$ map world→sensor; the inverse uses $\mathbf{R}^T$ (since $\mathbf{R}$ orthogonal $\Rightarrow \mathbf{R}^{-1} = \mathbf{R}^T$) and translation $-\mathbf{R}^T\mathbf{t} = \mathbf{C}$, the *sensor origin (camera center) in world coordinates*.

3.3.1 Motion Estimation from 3D and ICP Algorithm

Motion estimation from **3D point clouds** aims to find the 3D motion R, t of the sensor that best aligns two point clouds $\mathcal{P}_S, \mathcal{P}_{S'}$ obtained from the same static object (we get the movement relative to this point, ICP sensitive to such outliers — use robust Loss rather than default squared Euclidean norm).

Since we do not know the point correspondences we must apply some approximation.

Iterative Closest Point (ICP).

ICP solves the correspondence problem as an optimization. Given two sets of 3D points $\mathcal{P}_s = [\mathbf{P}_s^{(1)} \mathbf{P}_s^{(2)} \dots \mathbf{P}_s^{(n)}]$ and $\mathcal{P}_{s'} = [\mathbf{P}_{s'}^{(1)} \mathbf{P}_{s'}^{(2)} \dots \mathbf{P}_{s'}^{(n)}]$, find the 3D Euclidean motion $\{\hat{\mathbf{R}}, \hat{\mathbf{t}}\}$ and the assignment function $\hat{\phi} : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ that minimize the sum of squared errors:

$$\{\hat{\mathbf{R}}, \hat{\mathbf{t}}, \hat{\phi}\} = \arg \min_{\mathbf{R}, \mathbf{t}, \phi} E(\mathbf{R}, \mathbf{t}, \phi), \quad E(\mathbf{R}, \mathbf{t}, \phi) = \sum_{i=1}^n \left\| \mathbf{R}\mathbf{P}_s^{(\phi(i))} + \mathbf{t} - \mathbf{P}_{s'}^{(i)} \right\|_2^2,$$

where $\mathbf{P}_s^{(\phi(i))}$ and $\mathbf{P}_{s'}^{(i)}$ are the corresponding points induced by the assignment function ϕ . The two clouds need *not* be equal-sized: ϕ is a nearest-neighbour (many-to-one) map, not a bijection, so $n_s \neq n_{s'}$ is fine.

- **Key idea**: if the correct correspondences ϕ are *known*, the optimal $\{\hat{\mathbf{R}}, \hat{\mathbf{t}}\}$ has a **closed-form** solution.
- **Intuition**: at the optimal motion, corresponding points are closer to each other than to any other point.

- **Heuristic / caveat:** ϕ matches each point to its *currently* closest point $\Rightarrow$ a **good initialization** is required (converges to local minimum only).

Procedure (alternate assignment $\leftrightarrow$ motion until convergence):

1. Initial guess $\{\check{\mathbf{R}}, \check{\mathbf{t}}\}$ (usually I and 0).
2. *Assignment:* associate each point in $\mathcal{P}_{s'}$ with the closest point in $\mathcal{P}_s$ (defines ϕ ; need not be bijective $\Rightarrow$ no perfect matching of $\mathcal{P}_{s'}$ & $\mathcal{P}_s$ required - duplicate assignments in $\mathcal{P}_s$ possible).
3. *Motion:* solve the closed-form optimal $\{\hat{\mathbf{R}}, \hat{\mathbf{t}}\}$ for the fixed ϕ .
4. Repeat until convergence.

Optimal Motion.

Step (3) of finding the optimal motion $\{\hat{\mathbf{R}}, \hat{\mathbf{t}}\}$ given ϕ from Step (2) solved using **Least Squares Linear Regression**.

Caveat ($\mathbf{R} \notin$ vector space): translation $\mathbf{t}$ is plain least squares, but $\mathbf{R} \in SO(3)$ (special orthogonal group) has 9 params with only 3 DOF (entries interdependent) $\Rightarrow$ not amenable to default least squares. Strategy: *eliminate* $\mathbf{t}$ symbolically, reduce to a rotation-only problem, solve by SVD.

1. Optimal translation given rotation. Fix $\hat{\mathbf{R}}$. Each pair can be aligned exactly ($\hat{\mathbf{R}}\mathbf{P}_s^{(\phi(i))} + \mathbf{t} - \mathbf{P}_{s'}^{(i)} = 0$); stacking all n gives an overdetermined $3n \times 3$ system $\mathbf{A}\mathbf{t} = \mathbf{b}$ with $\mathbf{A} = [\mathbb{I}_3; \dots; \mathbb{I}_3]$, $\mathbf{b}^{(i)} = \mathbf{P}_{s'}^{(i)} - \hat{\mathbf{R}}\mathbf{P}_s^{(\phi(i))}$. Normal equations $\mathbf{t} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$ with $\mathbf{A}^T \mathbf{A} = n\mathbb{I}_3$ yield (a formula in $\mathbf{R}$, not yet a number):

$$\hat{\mathbf{t}} = \mu_{s'} - \hat{\mathbf{R}}\mu_{s,\phi}.$$

Intuition: the centroids of the two *matched* sets are aligned by $\{\hat{\mathbf{R}}, \hat{\mathbf{t}}\}$. Note $\mu_{s,\phi} \neq \mu_s$ in general (only ϕ -selected points contribute, ϕ not bijective).

2. Optimal rotation. Substitute $\hat{\mathbf{t}}(\mathbf{R})$ back into $E \Rightarrow \mathbf{t}$ cancels, objective is centered and depends on $\mathbf{R}$ only. Minimizing it $\Leftrightarrow$ maximizing

$$\max_{\mathbf{R}} \sum_{i=1}^n (\mathbf{P}_s^{(\phi(i))} - \mu_{s,\phi})^T \mathbf{R}^T (\mathbf{P}_{s'}^{(i)} - \mu_{s'}).$$

Writing $\mathbf{R}^T = \mathbf{U}\mathbf{V}^T$ (composition of two independent rotations) reduces this to $\max_{\mathbf{U}, \mathbf{V}} \text{trace}(\mathbf{U}^T \mathbf{W} \mathbf{V})$, where

$$\mathbf{W} = \sum_{i=1}^n (\mathbf{P}_s^{(\phi(i))} - \mu_{s,\phi}) (\mathbf{P}_{s'}^{(i)} - \mu_{s'})^T$$

is the **cross-covariance** of the two centered point sets.

3. Solve via SVD. The trace-maximization is solved exactly by the SVD $\mathbf{W} = \hat{\mathbf{U}}\mathbf{\Sigma}\hat{\mathbf{V}}^T$. To stay in $SO(3)$ (reject reflections $\rightarrow$ non-right-handed frames, which lie in $O(3) \setminus SO(3)$), correct with the determinants:

$$\hat{\mathbf{R}} = \hat{\mathbf{V}} \text{diag}(1, 1, \det(\hat{\mathbf{U}})\det(\hat{\mathbf{V}})) \hat{\mathbf{U}}^T.$$

Intuition: align the principal directions of variation (modeled by the cross-covariance) of the two corresponding sets.

Order note: the derivation eliminates $\mathbf{t}$ first to expose the rotation-only objective; at *runtime* compute $\hat{\mathbf{R}}$ (SVD) first, then $\hat{\mathbf{t}} = \mu_{s'} - \hat{\mathbf{R}}\mu_{s,\phi}$.

3.3.2 Motion Estimation from 2D and VO

We no longer leverage 3D Point Clouds to estimate motion but rather 2D images from various viewpoints.

Definition 13 (Visual Odometry.). *Assuming a single monocular system with images taken at time $k : I_{0:n} = \{I_0, \dots, I_n\}$, then two sequential images at $k-1$ and k are relate by the rigid body transformation:*

$$T_k = \begin{bmatrix} R_{k,k-1} & t_{k,k-1} \\ 0 & 1 \end{bmatrix}$$

Trough the concatenation of all transformations $T_{0:n}$ we can recover the full trajectory at each camera position $C_1, \dots, C_n$ relative to the initial coordinate frame C_0 (pose by pose/ 2D to 2D).

This works well given **sufficient illumination**, a **dominance of static** over dynamic scene constituents, **adequate texture** for informative feature extraction, and **sufficient co-visibility** between at least consecutive frame pairs.

3.3.3 Epipolar Geometry, Two-frame SFM, and the 8-Point Algorithm

Epipolar Geometry and Constraint

For a point p in $k - 1$ the matching point in k must lie on the *epipolar line* l' in the second image (**Epipolar Constraint**), where

- **epipolar line** l' : projection of the back-projected ray $\pi^{-1}(p) = \lambda K^{-1}p$ onto the camera image
- **epipole**: the projection of the optical center on the camera image; i.e., where the epipolar lines intersect (can be outside the image).
- **epipolar plane** is given by the hyperplane via the camera centers C_r, C_l and the image point

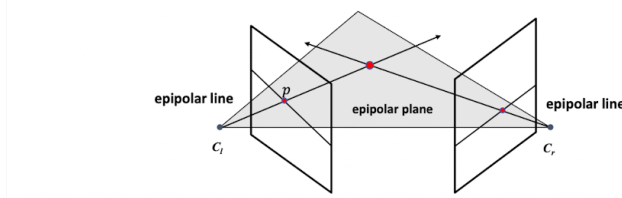


Figure 16: The Epipolar Constraint of Converging Cameras

- **Converging cameras**: As the position of the 3D point changes the epipolar lines rotate around the baseline
- **Identical and horizontally-aligned cameras (Simplified Case)**: e 's are at infinity, since the epipolar lines are essentially parallel - so the cameras never see each other no matter the size of b .
- **Sequential images in forward motion**: Epipolar with constant coordinates in both images (imaging the epipolar lines intersecting in the middle of the image at a focus point). The points move along these lines.

Essential Matrix

Given two normalized (homogeneous) image points $\bar{\mathbf{p}}_1, \bar{\mathbf{p}}_2$ and the relative pose $(R, \mathbf{T})$, we derive the essential matrix (3×3 ; 5DOF: 3 by R , 2 by t - translation up to scale only) from coplanarity of $\bar{\mathbf{p}}_2, \bar{\mathbf{p}}_1'$, and $\mathbf{T}$:

$$\begin{aligned}
 \bar{\mathbf{p}}_1' &= R \bar{\mathbf{p}}_1, \\
 \mathbf{n} &= \mathbf{T} \times \bar{\mathbf{p}}_1' \quad (\text{normal of the epipolar plane}), \\
 \bar{\mathbf{p}}_2^\top \mathbf{n} &= 0 \\
 \Rightarrow \bar{\mathbf{p}}_2^\top (\mathbf{T} \times (R \bar{\mathbf{p}}_1)) &= 0 \\
 \Rightarrow \bar{\mathbf{p}}_2^\top [\mathbf{T}]_\times R \bar{\mathbf{p}}_1 &= 0 \\
 \Rightarrow \bar{\mathbf{p}}_2^\top E \bar{\mathbf{p}}_1 &= 0, \quad E = [\mathbf{T}]_\times R.
 \end{aligned}$$

Intuition: $[\mathbf{T}]_\times$ turns “cross with $\mathbf{T}$ ” into a matrix, so E packages translation (up to scale) and rotation into one bilinear constraint that corresponding *normalized* points must satisfy.

8-Point Algorithm for the Essential Matrix.

Since each pair of *normalized* ($\tilde{\mathbf{p}}_1^{(i)} = \mathbf{K}^{-1} \mathbf{p}_1^{(i)}$ and zero-centered with std $\sqrt{2} \rightarrow \bar{\mathbf{p}}_1^{(i)}$) point correspondences provides a linear equation through the Epipolar constraint $\bar{\mathbf{p}}_2^\top E \bar{\mathbf{p}}_1 = 0$. Normalization is required since we have elementwise multiplication over columns that have different scales in orders of magnitude, that would lead to numerical instability:

$$\bar{u}_2 \bar{u}_1 e_{11} + \bar{u}_2 \bar{v}_1 e_{12} + \bar{u}_2 e_{13} + \bar{v}_2 \bar{u}_1 e_{21} + \bar{v}_2 \bar{v}_1 e_{22} + \bar{v}_2 e_{23} + \bar{u}_1 e_{31} + \bar{v}_1 e_{32} + e_{33} = 0$$

We can stack these equations for n correspondences into a single homogeneous linear system

$$Q \bar{\mathbf{E}} = \mathbf{0}, \quad \bar{\mathbf{E}} = \text{vec}(E) = [e_{11} \ e_{12} \ e_{13} \ e_{21} \ e_{22} \ e_{23} \ e_{31} \ e_{32} \ e_{33}]^\top,$$

where each row of $Q \in \mathbb{R}^{n \times 9}$ is built from one correspondence $(\bar{u}_1, \bar{v}_1) \leftrightarrow (\bar{u}_2, \bar{v}_2)$:

$$Q_i = [\bar{u}_2 \bar{u}_1 \ \bar{u}_2 \bar{v}_1 \ \bar{u}_2 \ \bar{v}_2 \bar{u}_1 \ \bar{v}_2 \bar{v}_1 \ \bar{v}_2 \ \bar{u}_1 \ \bar{v}_1 \ 1].$$

Minimal case ($n = 8$). Each correspondence contributes one independent linear constraint, so 8 points are sufficient to determine $\bar{\mathbf{E}}$ up to scale (in practice Q should have rank 8 so that $\text{null}(Q)$ is 1D).

Over-determined case ($n > 8$). Solve in least-squares form

$$\min_{\|\bar{\mathbf{E}}\|=1} \|Q\bar{\mathbf{E}}\|^2,$$

whose solution is the right singular vector of Q associated with the smallest singular value (equivalently, the eigenvector of $Q^\top Q$ with smallest eigenvalue). Reshape $\bar{\mathbf{E}}$ into $E \in \mathbb{R}^{3 \times 3}$.

Enforcing the essential-matrix structure (rank-2 constraint). Because $E = [\mathbf{T}]_\times R$, a true essential matrix has *two equal non-zero singular values and one zero* (so $\text{rank}(E) = 2$, 5 DOF). The linear estimate $\hat{E}$ from noisy data generally violates this, so we project it onto the closest valid essential matrix with a *second* (now only 3×3) SVD, averaging the two largest singular values and forcing the smallest to zero:

$$\hat{E} = U_E \Sigma_E V_E^\top \longrightarrow E^* = U_E \text{diag}\left(\frac{\sigma_E^{(1)} + \sigma_E^{(2)}}{2}, \frac{\sigma_E^{(1)} + \sigma_E^{(2)}}{2}, 0\right) V_E^\top.$$

Undoing the normalizations. Finally reverse the pre-processing in the opposite order: since $\bar{\mathbf{p}} = N \tilde{\mathbf{p}}$ turns the constraint into $\tilde{\mathbf{p}}_2^\top (N_2^\top E^* N_1) \tilde{\mathbf{p}}_1 = 0$, apply the inverse linear transforms of the $\text{std-}\sqrt{2}$ conditioning (step 2) to map E^* back to the original normalized-image coordinates.

Degenerate configurations. The linear 8-point estimation becomes ill-conditioned/degenerate when the underlying 3D points are (nearly) coplanar; in such cases alternative minimal solvers (e.g. 5-point methods) are preferred.

Recovering $(R, \mathbf{T})$ — the 4 Solutions.

A final 3×3 SVD $E^* = U_E \Sigma_{E^*} V_E^\top$ decomposes the essential matrix into motion. Because the sign of the translation and the direction of rotation interfere, this yields **four** candidate motions, built from the third column $\mathbf{u}_3$ of U_E and the fixed orthogonal matrix

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} : \quad \mathbf{T} = \pm \mathbf{u}_3, \quad R = U_E W V_E^\top \quad \text{or} \quad U_E W^\top V_E^\top.$$

Any candidate R with $\det R = -1$ is negated to make it a proper rotation ($\det R = +1$). The single physically valid motion is selected by **triangulating** the correspondences and keeping the solution for which the reconstructed 3D points lie *in front of both cameras* ($z > 0$, the cheirality check). This **gives us $\mathbf{T}$ up to scale**.

2D-to-2D VO Pipeline.

One iteration of the monocular VO loop: (1) capture I_k ; (2) match features $I_{k-1} \leftrightarrow I_k$; (3) estimate E (8-point); (4) decompose into $R_k, \mathbf{t}_k$, form T_k ; (5) compute relative scale and rescale $\mathbf{t}_k$; (6) concatenate the global pose $C_k = C_{k-1} T_k$; repeat.

Relative Scale.

The absolute scale is unrecoverable from images, but the *relative* scale between consecutive transforms is enough for a consistent trajectory. Triangulate the same points from two subsequent pairs (X_{k-1}, X_k) ; the scale is the distance ratio over co-observed points i, j (averaged/RANSAC'd in practice):

$$r = \frac{\|X_{k-1,i} - X_{k-1,j}\|}{\|X_{k,i} - X_{k,j}\|}.$$

3.3.4 RANSAC

Sensor noise, image blur, algorithmic noise (like greedy algos) etc, we must do outlier rejection via *robust estimation*. **RANSAC** is the standard approach for fitting a model in the presence of many outliers (e.g. E).

- **Model in SfM:** E (calibrated) (or F if uncalibrated); equivalently (R, T) once calibrated.
- **Minimal sample size:** depends on the solver (e.g. $m=8$ for the classic 8-point E).
- **Distance to the model:** choose a residual $r_i(\theta)$, e.g. some geometric or (symmetric) epipolar line distance
- **Idea:** fit many hypotheses from random minimal samples and keep the one with the largest inlier set.
- **Loop:** sample m points $\rightarrow$ fit $\theta \rightarrow$ compute residuals $\rightarrow$ inliers if $r_i(\theta) < \tau$.
- **Output:** best hypothesis + **refit** using all inliers (least squares).
- **Notes:** non-deterministic; robust to bad initialization; quality depends on τ , m , and k .

Required Iterations.

For inlier fraction w , sample size m , and success probability p , the iterations $k = \frac{\log(1-p)}{\log(1-w^m)}$ grow fast with the outlier rate and m (e.g. $w = 50\%, p = 99\%$: $m = 8 \rightarrow k \approx 1177$ vs. $m = 2 \rightarrow k \approx 16$). Hence smaller minimal samples via motion/scene constraints (planar $\rightarrow$ 2-point, vehicle $\rightarrow$ 1-point) drastically cut cost.

3.3.5 Learned Feature Mapping and SuperGlue

Approaches:

- **Nearest-neighbor matching:** compare individual descriptors and link closest pairs — greedy, locally optimal, ignores assignment structure.
- **Learned matching (SuperGlue):** cast both objectives as one *differentiable global optimization* over all features in both frames, solved end-to-end rather than per-keypoint.

SuperGlue: Differentiable Optimal Matching.

Since each keypoint matches at most one (or none), the matching is a **partial assignment** between the keypoint sets $\mathcal{A} = \{1, \dots, M\}$ and $\mathcal{B} = \{1, \dots, N\}$. SuperGlue makes this *soft and differentiable*: every possible pair (i, j) gets a **confidence** $p_{ij} \in [0, 1]$ computed from the descriptors $\mathbf{f}_i^{(1)}, \mathbf{f}_j^{(2)}$ ($p_{ij} = 1$ = certain match). The partial-assignment property is enforced by the constraints

$$\sum_{j=1}^N p_{ij} \leq 1 \quad \forall i \in \mathcal{A}, \quad \sum_{i=1}^M p_{ij} \leq 1 \quad \forall j \in \mathcal{B},$$

i.e. each keypoint distributes at most unit confidence (the slack < 1 absorbs unmatched keypoints).

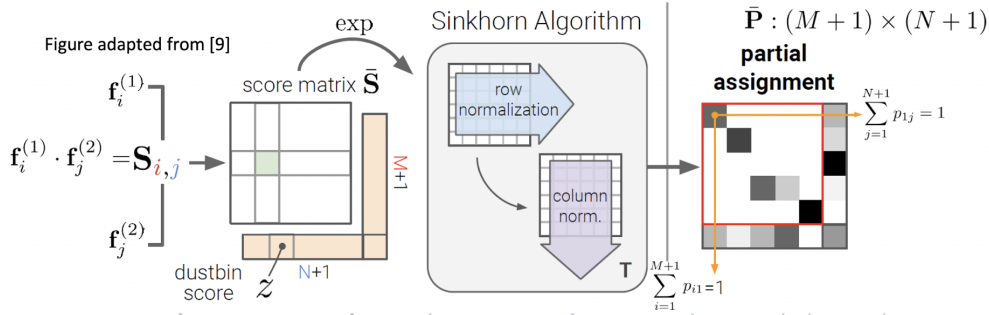


Figure 17: SuperGlue Differentiable Optimal Matching

Pipeline (Fig. 17):

1. **Score matrix.** $\mathbf{S}_{i,j} = \mathbf{f}_i^{(1)} \cdot \mathbf{f}_j^{(2)}$ ($M \times N$): dot-product similarity, more similar descriptors $\Rightarrow$ larger score. $\rightarrow$ raw, unnormalized affinities, not yet valid confidences.
2. **Dustbin augmentation.** Append one row & column with a *learnable shared* score z , giving $\tilde{\mathbf{S}}$ of size $(M+1) \times (N+1)$. The dustbin soaks up unmatched keypoints, turning the ≤ 1 constraints into equalities (each row/column, dustbin included, sums to 1 $\Rightarrow$ normalization). $\rightarrow$ a “trash can” that lets a row-sum still reach 1 when a keypoint has no real partner; this is what allows non-matching.
3. **Sinkhorn.** Exponentiate $\tilde{\mathbf{S}}$, then iterate alternating row- and column-normalization for T steps. This converges to a doubly-stochastic soft assignment $\bar{\mathbf{P}}$ that satisfies the constraints differentiably. $\rightarrow$ fixing rows breaks columns and vice versa, so alternating converges to satisfy both; a soft, gradient-friendly stand-in for the hard Hungarian assignment.
4. **Output.** $\bar{\mathbf{P}} : (M+1) \times (N+1)$ partial assignment; drop the dustbin row/column to read off matches. $\rightarrow$ confidence left in the dustbin means “no match”.
5. **Training.** Maximum-likelihood loss $L = -\sum_{(i,j) \in \mathcal{M}} \log \mathbf{P}_{ij}$ over GT (non-)matches $\mathcal{M}$, back-propagated into the learnable descriptors. $\rightarrow$ rewards high probability on true matches, shaping descriptors so correct pairs score highest.

Input: Attentional Context Aggregation (how do we get the context-aware descriptors $\mathbf{f}_i^{(1)}, \mathbf{f}_j^{(2)}$ fed to the optimal matching). Raw local descriptors (e.g. SIFT) are purely *local*; matching needs descriptors **aware of the global context** (spatial layout in both frames). Achieved with attention, treating each keypoint as a **token**:

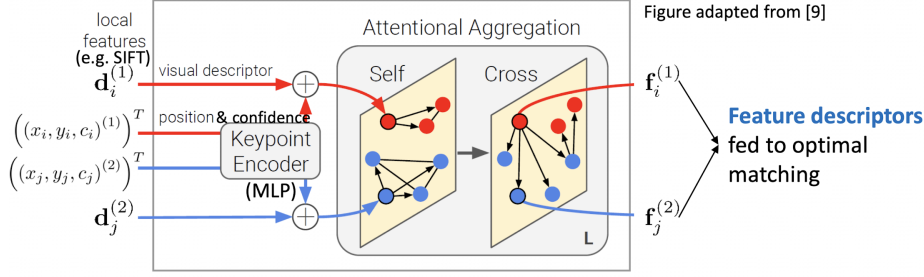


Figure 18: SuperGlue Attentional Context Aggregation

1. **Keypoint Encoder.** MLP embeds position & confidence (x_i, y_i, c_i) and *adds* it to the visual descriptor $\mathbf{d}_i$, giving the initial embedding $\mathbf{x}_i^{(0)}$. $\rightarrow$ fuses *appearance* + *location* into one token.
2. **Alternating attention (L layers).** Residual attention block $\mathbf{X}^{(l)} = \mathbf{X}^{(l-1)} + \text{MLP}(\text{softmax}(\mathbf{Q}^\top \mathbf{K}) \mathbf{V})$, with $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ learned projections of the embeddings:
 - **Self-attention** (odd layers): $\mathbf{K}, \mathbf{V}$ from the *same* image $\rightarrow$ attend to adjacent/salient/similar-looking keypoints (intra-frame layout).
 - **Cross-attention** (even layers): $\mathbf{K}, \mathbf{V}$ from the *other* image $\rightarrow$ attend to keypoints near the actual match (inter-frame comparison). $\rightarrow$ cascading both makes each keypoint aware how its neighbors and salient keypoints map to the other image.
3. **Final projection.** Linear layer $\mathbf{f}_i^{(1)} = \mathbf{W} \mathbf{x}_i^{(2L)}$ yields the context-aware descriptors fed to the matching stage.

3.3.6 Neural Radiance Fields (NeRF)

Why: Classical 3D reps each lose something: point clouds / meshes / voxels are **discrete** (info loss); multi-view images are only 2D projections; implicit functions need GT geometry. **NeRF:** store the scene as *one continuous function* (the MLP weights), learned from *posed images alone* (no GT geometry). $\rightarrow$ effectively infinite resolution from a few MB of weights; goal is **novel-view synthesis** (render the scene from unseen camera poses).

Core idea: $F_\Theta : \underbrace{(x, y, z)}_{\text{position}} + \underbrace{(\theta, \phi)}_{\text{view dir.}} \mapsto \underbrace{\sigma}_{\text{density}} + \underbrace{(r, g, b)}_{\text{radiance}}$. Density = *view-independent geometry* (does a surface exist here?); radiance = *view-dependent color* (a glossy highlight shifts as you move, the surface does not, hence the split). Render via volume rendering $\rightarrow$ fully differentiable $\rightarrow$ train on a nixel reconstruction loss.

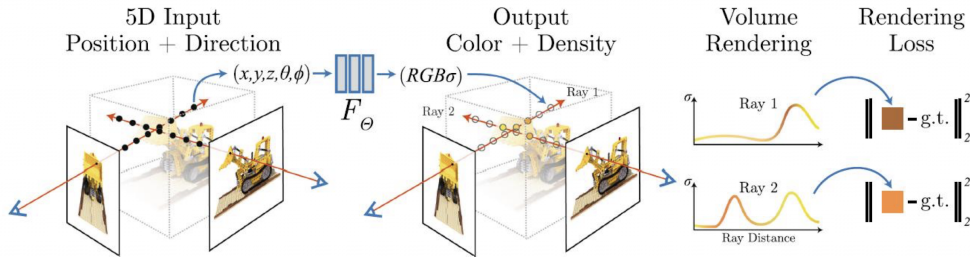


Figure 19: Neural Radiance Fields

Architecture trick (enforces view-independent σ): $\text{MLP}_1(\mathbf{x}) \rightarrow \sigma + \text{feature } \mathbf{f}$; then append (θ, ϕ) to $\mathbf{f} \rightarrow \text{MLP}_2 \rightarrow \mathbf{c}$. So geometry cannot change with viewpoint by construction; only color can. $\rightarrow$ prevents the net from “cheating” by faking geometry per view, which would break multi-view consistency.

Volume rendering (how the MLP becomes a pixel): read $\sigma(\mathbf{x})$ as the **probability density of ray termination** at $\mathbf{x}$ (likelihood a photon is absorbed/reflected there). A pixel is the expected radiance along its ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$:

$$\mathbf{C}(\mathbf{r}) = \int_{t_n}^{t_f} \underbrace{T(t)}_{\text{reach } t} \underbrace{\sigma}_{\text{stop here}} \mathbf{c} dt, \quad T(t) = \exp\left(-\int_{t_n}^t \sigma ds\right) \text{ (transmittance)}.$$

Intuition: T = chance the ray survived this far; at a surface σ spikes $\Rightarrow T$ collapses $\Rightarrow$ the first opaque hit dominates the integral $\Rightarrow$ occlusion is handled for free, and empty space ($\sigma \approx 0$) simply contributes nothing.

Discretize & train: N stratified (jittered) samples per ray turn the integral into a weighted sum, segment opacity $\alpha_i = 1 - e^{-\sigma_i \delta_i}$:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^N T_i \alpha_i \mathbf{c}_i, \quad T_i = \prod_{j < i} (1 - \alpha_j), \quad \delta_i = t_{i+1} - t_i.$$

This is just *alpha compositing* of the samples front-to-back, and it is differentiable in $(\sigma_i, \mathbf{c}_i) \Rightarrow$ the pixel loss back-props through rendering into F_Θ (no 3D supervision needed). *Jittering* the samples avoids the net overfitting to fixed depths.

Positional encoding (why detail isn't blurry): MLPs are *spectrally biased* to low frequencies, so raw $xyz\theta\phi$ inputs $\Rightarrow$ smooth/blurry output. Map each coord through fixed sinusoids of exponentially growing frequency, $F_\Theta = F'_\Theta \circ \gamma$:

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p)), \quad L=10 \text{ (x)}, L=4 \text{ (d)}.$$

Intuition: nearby coords get very different encodings at high frequencies, handing the network a high-frequency basis (Fourier features) $\Rightarrow$ it can represent sharp edges & fine texture instead of averaging them away.

Limitations

Basic NeRF is **per-scene** (one trained network = one scene, no generalization), covers only a **bounded volume**, and assumes a **static scene**: fixed illumination/medium density, no transient or dynamic objects. $\rightarrow$ exactly the assumptions a *driving* scene violates (photos span hours/seasons, cars & pedestrians move, cities are huge), motivating the two extensions below.

Extensions for Automated Driving

The driving setting breaks NeRF's static + bounded assumptions, so two ideas adapt it:

- **NeRF in the Wild** (handles *appearance* & *transient variation*): splits the scene into a **static** branch and a **transient** branch. Geometry (density) stays shared, but color is conditioned on a per-image **appearance embedding** (absorbs changing illumination/weather across photos) and a separate transient head with an **uncertainty** β models dynamic objects so they don't corrupt the static reconstruction. $\rightarrow$ lets NeRF train on uncontrolled, time-varying captures instead of one clean static scene.
- **Block-NeRF** (handles *scale*): a single MLP can't fit a city, so the area is tiled into **many overlapping Block-NeRFs**, one per intersection. Dynamic objects are **masked out** (semantic segmentation) during training, and each network also predicts **visibility** (transmittance of a ray). At render time, neighboring blocks are **composited** by inverse-distance weighting (low-visibility blocks discarded) $\rightarrow$ smooth transitions and **city-scale**, reconstruction from real driving logs.

Takeaway: together they remove the static-scene and bounded-volume limitations $\Rightarrow$ NeRF becomes usable for large, dynamic, real-world driving environments (simulation, mapping, scene reconstruction).

4 Part IV: Robust Perception: Multi-Modal, Multi-Domain and Multi-Agent Methods

4.1 Domain Adaptation

4.1.1 Motivation, Taxonomy, and Fundamentals

For SAE level 5 (full automation) we require *unlimited* Operation Design Domain, by definition — in normal and adverse conditions (generalization is often poor with domain shift).

Domain Shift Formal Definition

A **domain** is the joint distribution $p(I, Y)$ of inputs I and reference outputs Y . Source samples $(I_s, Y_s) \sim p_s$, target samples $(I_t, Y_t) \sim p_t$. **Domain shift**: in general $p_s \neq p_t$ for (unseen) target data $\rightarrow$ models trained on p_s generalize poorly to p_t .

Domain Shifts Categorized by what changes between source and target domain:

- **Low-level** (appearance only like color, semantics unchanged): *condition-level* shift (weather, day/night) and *sensor* differences (camera model/optics).
- **High-level** (semantics/structure): *semantic* shift in object size, shape, or spatial configuration, e.g. by geographic location.
- **Both**: *synthetic vs. real* (rendering) combines a low-level (look) and high-level (layout) shift.

Why not just train on everything?

Naïve: add all “outsider” domains to labeled training set and large enough foundation model $\rightarrow$ basically impossible.

- **Curse of dimensionality**: must cover *all combinations* of shift factors (weather $\times$ sensor $\times$ location $\times \dots$).
- **Open domain**: test data can still be unseen/rare/unpredicted domain $\rightarrow$ unlimited ODD must handle it.
- **Annotation cost**: exhaustive labels for massive data are expensive, especially for dense tasks (segmentation).
- **Acquisition**: collecting data for narrow domains is hard: geographies, adverse conditions, dangerous collection.

Task Variants

Distinguished by which assets are available and *when* (here: semantic segmentation):

- **Unsupervised DA (UDA)**: labeled source + *unlabeled* target, at train time.
- **Weakly-supervised DA**: as UDA, but target has **correspondences** to source (e.g. same place day/night).
- **Source-free / model adaptation**: only a *pretrained source model* (no source data) + unlabeled target.
- **Test-time / online DA**: pretrained model adapted on the fly to **unlabeled sequential test data**.
- **Domain generalization**: (large) labeled source(s) only; target **unseen**, *no* adaptation at test time.

Levels of Adaptation

Orthogonally to the task variants above, alignment can be enforced at different **levels** of the network — which is how the methods below are organized: **input** (align image distributions) $\rightarrow$ **feature** (align feature maps) $\rightarrow$ **output** (align predictions) $\rightarrow$ **test-time** (adapt online during inference).

4.1.2 Input-Level Adaptation

Align the *image distributions of the source and the target domain*: for *low-level* domain shifts (e.g. fog/no fog).

Physics-based Domain Translation

Leverage known priors of the domain shift: no target needed, reliable, controllable, and interpretable; but hard or impossible: assumptions necessary.

Image-Based: e.g. fog/rain simulation via depth plane fitting and guided filters using (like NeRF) $\hat{t}(x) = e^{-\alpha l(x)}$ with attenuation coefficient α to control fog density. Labels transfer for free:

- If source is *labeled*: synthesize the target style and **keep the source labels** (no real target data needed).
- If source is *unlabeled*: run a segmentation model on it and use those predictions as **pseudo ground truth** for the synthesized target images.

Lidar-Based: e.g. snowfall has particles, so no longer just attenuation (as in the clear-air lidar introduction) but also *backscattering*. Idea: simulate small spheres in the medium as snowflakes, per lidar pulse (small opening angle, azimuth $\approx 0.1^\circ$). Each simulated particle j intersects the beam with a distribution-driven diameter D_j , range R_j , and center; from these its **occlusion angle** θ_j follows, partitioning the beam as $\theta = \theta_0 + \sum_{j=1}^n \theta_j$ (θ_0 : un-occluded part reaching the target).

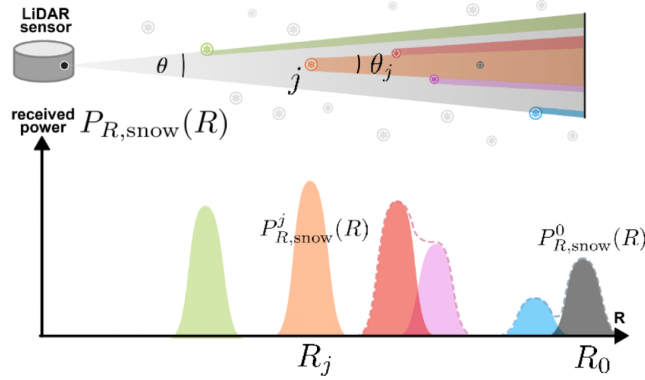


Figure 20: Pulsed Lidar in Snowfall

The *target* response must also account for the snowflakes: since a flake is smaller than the pulse extent ($D_j \ll c\tau_H$), its impulse response is modeled as a Dirac delta. A flake's peak can exceed the real object's $\rightarrow$ snow-caused noisy (spurious) lidar points.

Snowfall — target impulse response. The linear system model and optical channel $H_C(R) = \frac{T^2(R)}{R^2}\xi(R)$ are *unchanged* (same as clear weather); only the *target* response H_T changes. The beam (opening angle θ) is partly occluded by n flakes, each blocking a sub-angle θ_j at range R_j , so $\theta = \theta_0 + \sum_{j=1}^n \theta_j$ (θ_0 : fraction still reaching the real target). With $D_j \ll c\tau_H$ each flake is a Dirac in range:

$$H_T(R) = \frac{1}{\theta} \left(\underbrace{\rho_0 \theta_0 \delta(R - R_0)}_{\text{real target}} + \rho_s \sum_{j=1}^n \underbrace{\theta_j \delta(R - R_j)}_{\text{flake } j} \right) = H_T^0(R) + \sum_{j=1}^n H_T^j(R),$$

i.e. a superposition weighted by each return's occlusion fraction θ_j/θ and reflectivity (ρ_s snow, ρ_0 target).

Snowfall — received waveform. By linearity of the convolution the received power is the same superposition,

$$P_{R,\text{snow}}(R) = P_{R,\text{snow}}^0(R) + \sum_{j=1}^n P_{R,\text{snow}}^j(R),$$

and substituting H_T^j into the linear system model shifts/scales the emitted $\sin^2$ pulse to range R_j :

$$\begin{aligned} P_{R,\text{snow}}^j(R) &= \frac{C_A P_0 \rho_s \theta_j \xi(R_j)}{\theta R_j^2} \int_0^{2\tau_H} \sin^2\left(\frac{\pi}{2\tau_H} t\right) \delta\left(R - \frac{ct}{2} - R_j\right) dt \\ &= \begin{cases} \frac{C_A P_0 \rho_s \theta_j \xi(R_j)}{\theta R_j^2} \sin^2\left(\frac{\pi(R - R_j)}{c\tau_H}\right), & R_j \leq R \leq R_j + c\tau_H, \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

and analogously $P_{R,\text{snow}}^0$ for the real target at R_0 (with ρ_0, θ_0).

Intuition: each flake reflects back the beam fraction θ_j/θ it occludes $\rightarrow$ one $\sin^2$ peak per flake at its range R_j . Near flakes have small R_j , so the $1/R_j^2$ gain (plus high snow reflectivity ρ_s) can make a clutter peak *exceed* the true target peak $\rightarrow$ spurious foreground points, while the target only receives the un-occluded fraction θ_0/θ (weakened/lost return).

Learned Domain Translation

Data-driven domain translation: requires target data, can learn complex shifts but unreliable, not very controllable nor interpretable. I.e., use a NN (image translator) to translate the source image into data representing the target domain.

Paired Data / Supervised.

Works primarily when we have *paired data*. Often misaligned pairs in reality (e.g. we can have driving data from the same location at different day times, yet the camera is not exactly the same, and dynamic objects are different).

Unpaired Data / Unsupervised.

Only two *unaligned* sets X_1 (e.g. day), X_2 (e.g. night) — *no* correspondence $x_i \leftrightarrow y_i$. Realistic for driving (can't capture the exact same scene at two times of day).

GANs. **Generator** produces fake samples (from noise), **Discriminator** learns to tell real vs. fake $\rightarrow G$ is pushed to match the target distribution.

Conditional GAN for translation. Feed a source image instead of noise: a **translation net** $F_{1 \rightarrow 2}$ maps $X_1 \rightarrow X_2$, D enforces outputs *look like* domain 2. Only needs a domain-2 set, no pairs (e.g. translate dogs to cats).

Mode Collapse. GANs only force $F(x) \in X_2$, *not* that the output **corresponds** to the input $\rightarrow F$ can map all day images to one generic night image and still fool D (scene content lost).

CycleGAN. Add a **cycle-consistency loss**: round-trip $x \xrightarrow{G} G(x) \xrightarrow{F} F(G(x))$ must reconstruct the input, $\|F(G(x)) - x\|_1$ small. *Intuition:* ties output to input **without** paired data $\rightarrow$ kills mode collapse (a collapsed mapping can't be inverted). E.g. **time-of-day translation** (day $\leftrightarrow$ twilight $\leftrightarrow$ night) to synthesize labeled adverse-condition data.

Hand-crafted Domain Transformation

Transform in a fixed, *non-learned* space (e.g. frequency): needs a target reference image; **controllable + interpretable**, but limited to the attributes of that one image and not always reliable.

Fourier Domain Adaptation (FDA).

Observation: high-level object attributes (shape of cars, humans) barely change across domains (low-frequency shift). **Hypothesis:** aligning only the *low-level statistics* of source to target is a good UDA baseline: FDA does this at the **spectral level/ frequency decomposition**, non-learned.

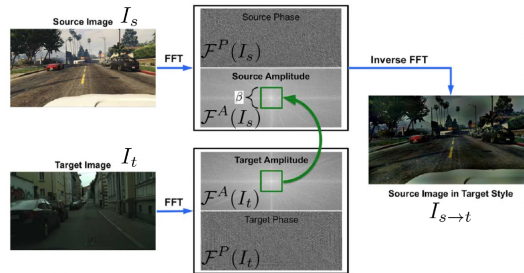


Figure 21: Fourier Domain Adaptation

Method. Split the DFT into amplitude and phase, $\mathcal{F}(I) = \mathcal{F}^A(I) e^{j\mathcal{F}^P(I)}$. Low-level statistics live in the *low-frequency amplitude* (near the center in fourier/ zero frequency), so **swap the source's low-freq amplitude for the target's**, keep the source *phase* (= content). With low-pass mask $M_\beta(h, w) = \mathbf{1}_{(h,w) \in [-\beta H: \beta H, -\beta W: \beta W]}$ (replace the low-level frequencies):

$$I_{s \rightarrow t} = \mathcal{F}^{-1} \left(\left[M_\beta \mathcal{F}^A(I_t) + (1 - M_\beta) \mathcal{F}^A(I_s), \mathcal{F}^P(I_s) \right] \right)$$

Bandwidth β : $\beta = 0 \Rightarrow$ output = source; larger $\beta \Rightarrow$ closer to target *but* more artifacts. *Intuition:* cheap, interpretable "style transfer" $\rightarrow$ recolor the source to the target's look without touching structure. Bonus: varying β yields **multiple diverse models** (one training set per β), which learned translators don't give for free.

4.1.3 Feature-Level Adaptation

Align the *feature distributions of the source and the target domain*: impose constraints/restrict feature maps.

CyCADA (Cycle-Consistent Adversarial Domain Adaptation).

Combines *input-level* (CycleGAN-style image translation) **and** *feature-level* alignment, while preserving semantics.

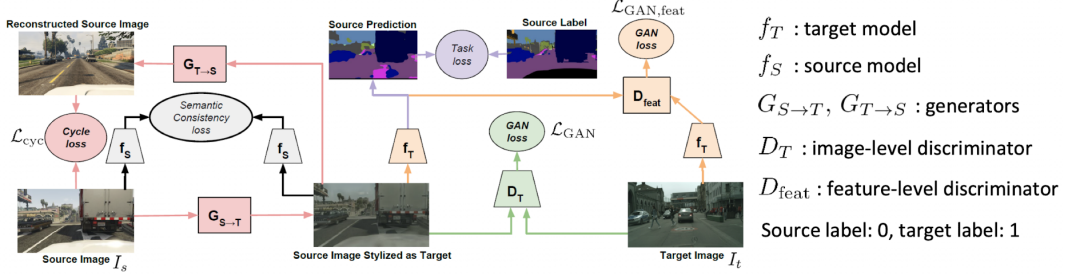


Figure 22: Cycle-Consistent Adversarial Domain Adaptation

Input-level adaptation. Translate source $\rightarrow$ target style so a translated image is *indistinguishable* from a real target image (adversarial), with cycle-consistency: ($\mathcal{L}_{GAN}$ minimized by generator, maximized by discriminator.)

$$\begin{aligned}\mathcal{L}_{GAN} &= \mathbb{E}_{I_t}[\log D_T(I_t)] + \mathbb{E}_{I_s}[\log(1 - D_T(G_{S \rightarrow T}(I_s)))] \\ \mathcal{L}_{cyc} &= \mathbb{E}_{I_s}[\|G_{T \rightarrow S}(G_{S \rightarrow T}(I_s)) - I_s\|_1] \text{ (+ target dir.)}\end{aligned}$$

Semantic consistency. The source model must give the *same predictions* on the translated image as on the original $\rightarrow$ **car stays car, road stays road** (translation can't destroy labels).

Feature-level adaptation. A second adversarial game on *features*: D_{feat} can't tell a translated image's representation $f_T(G_{S \rightarrow T}(I_s))$ from a real target one $f_T(I_t)$.

$$\mathcal{L}_{GAN, feat} = \mathbb{E}_{I_t}[\log D_{feat}(f_T(I_t))] + \mathbb{E}_{I_s}[\log(1 - D_{feat}(f_T(G_{S \rightarrow T}(I_s)))]$$

Intuition: alignment at *both* pixel and feature level $\rightarrow$ what's left of the domain gap after recoloring is closed in feature space, semantic consistency stops the GAN from cheating by changing content.

4.1.4 Output-Level Adaptation

Align the *outputs for the source and the target domain*, i.e., at the prediction level.

AdaptSegNet: Adversarial Discriminative Domain Adaptation.

Observation: the segmentation output space is *structured* (layout, shape, spatial relations are shared across domains, even if appearance differs); softmax maps have *low entropy*/ low variation (locally) on the supervised source.

Idea: adapt at the *output level*: make target predictions *indistinguishable* from source ones. Vs. CyCADA: adversarial learning is (1) *directly on the output/prediction space* (not features), via (2) a *pixel-level discriminator* on segmentation maps.

Setup. One *shared* segmentation net f produces softmax maps $P_s = f(I_s)$, $P_t = f(I_t)$; a discriminator D outputs *dense* per-pixel domain labels (source = 0, target = 1). Two losses train f : the standard segmentation cross-entropy $\mathcal{L}_{seg}$ on labeled source, and an adversarial term.

$$\begin{aligned}\mathcal{L}_{adv, D} &= \mathbb{E}_{I_s} \left[\sum_{h,w} \log(1 - D(P_s)_{(h,w)}) \right] + \mathbb{E}_{I_t} \left[\sum_{h,w} \log D(P_t)_{(h,w)} \right] \\ \mathcal{L}_{adv, f} &= \mathbb{E}_{I_t} \left[\sum_{h,w} \log D(P_t)_{(h,w)} \right]\end{aligned}$$

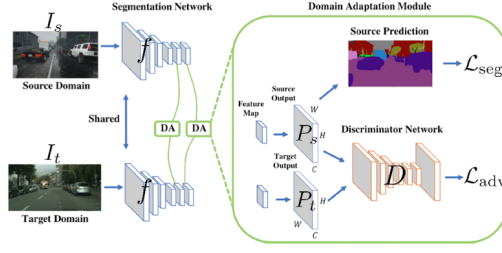


Figure 23: AdaptSegNet

Objective: f minimizes $\mathcal{L}_{\text{seg}}$ on source *while* fooling D (target predictions classified as source); D maximizes domain separation. *Intuition:* no labels in target, but forcing target output maps to look source-like transfers the learned, well-structured layout to the target domain.

Self-Training Strategies (Pseudo-Labeling) in Target Domain

Idea: use the model's own confident target predictions as **pseudo-labels** $\hat{y}$, retrain on them, repeat. Notation: s/t index source/target images, n pixels, $p_n(w, I)$ the softmax, y one-hot GT. The three cases below are the *same* cross-entropy, differing only in what plays the role of the target label.

1) Supervised (the ideal, unavailable). If we *had* target labels y_t , just do cross-entropy on both domains:

$$\min_w \mathcal{L}_S(w) = - \sum_{s,n} y_{s,n}^\top \log p_n(w, I_s) - \sum_{t,n} y_{t,n}^\top \log p_n(w, I_t).$$

2) Default self-training. Replace the missing y_t by pseudo-labels $\hat{y}_t$ and *jointly* optimize weights **and** pseudo-labels - we only have labels for the source domain:

$$\min_{w, \hat{y}} \mathcal{L}_U(w, \hat{y}) = - \sum_{s,n} y_{s,n}^\top \log p_n(w, I_s) - \sum_{t,n} \hat{y}_{t,n}^\top \log p_n(w, I_t), \quad \text{s.t. } \hat{y}_{t,n} \in \{e^{(i)}\}.$$

Constraint: each $\hat{y}_{t,n}$ is a **one-hot** vector $e^{(i)} \in \mathbb{R}^C$ (a hard class). *Problem:* *every* pixel is forced to take a label, so low-confidence/wrong pseudo-labels get reinforced.

3) Curriculum self-training. Two changes let the model **abstain** on uncertain pixels:

$$\min_{w, \hat{y}} \mathcal{L}_{ST} = - \sum_{s,n} y_{s,n}^\top \log p_n(w, I_s) - \sum_{t,n} \left[\hat{y}_{t,n}^\top \log p_n(w, I_t) + k \|\hat{y}_{t,n}\|_1 \right], \quad \hat{y}_{t,n} \in \{e^{(i)}\} \cup \{\mathbf{0}\}, \quad k > 0.$$

(i) the pseudo-label may now be the **zero vector** $\mathbf{0}$ (= don't label this pixel); (ii) the regularizer $k\|\hat{y}_{t,n}\|_1$ *rewards abstaining* ($\|\hat{y}\|_1 = 1$ if labeled, 0 if not) $\rightarrow$ a pixel is only labeled if it pays off the penalty k . **Solved by alternating block coordinate descent** (fix one block, optimize the other):

Step a) pseudo-label update (w fixed). Drop the (constant) source term; only the target term depends on $\hat{y}$:

$$\min_{\hat{y}} - \sum_{t,n} \left[\sum_{c=1}^C \hat{y}_{t,n}^{(c)} \log p_n(c | w, I_t) + k \|\hat{y}_{t,n}\|_1 \right], \quad \hat{y}_{t,n} = [\hat{y}_{t,n}^{(1)}, \dots, \hat{y}_{t,n}^{(C)}] \in \{e^{(i)}\} \cup \{\mathbf{0}\}.$$

This **decomposes over pixels** (each (t, n) is independent) $\rightarrow$ the optimum is a **trivial per-pixel closed form**: take the most confident class $c^* = \arg \max_c p_n(c)$ and

$$\hat{y}_{t,n} = \begin{cases} e^{(c^*)}, & p_n(c^*) > e^{-k} \text{ (confident enough)}, \\ \mathbf{0}, & \text{otherwise (abstain)}. \end{cases}$$

Derivation: per pixel, labeling with c^* costs $-\log p_{c^*} - k$, abstaining ($\hat{y} = \mathbf{0}$) costs 0; label iff $-\log p_{c^*} - k < 0 \Leftrightarrow p_{c^*} > e^{-k}$.

Step b) network update ($\hat{y}$ fixed). Standard supervised cross-entropy training on the source GT **plus** the admitted target pseudo-labels.

Curriculum = schedule of k . k sets the confidence threshold e^{-k} for **admitting** pseudo-labels: small $k \Rightarrow$ high bar (few, very confident labels); growing k over rounds lowers the bar to admit progressively more (harder) pixels — **easy-to-hard**, which is the curriculum.

Class-balanced self-training (CBST). *Problem:* a single k biases pseudo-labels towards **easily transferable classes** (e.g. road/sky) whose target predictions are systematically more confident $\rightarrow$ rare/hard classes never get labeled. *Fix:* a **per-class** threshold k_c (class-adaptive regularization), replacing the single penalty $k\|\hat{y}_{t,n}\|_1$ by the per-class $\sum_c k_c \hat{y}_{t,n}^{(c)}$:

$$\min_{w, \hat{y}} \mathcal{L}_{CB} = - \sum_{s,n} y_{s,n}^\top \log p_n(w, I_s) - \sum_{t,n} \sum_{c=1}^C \left[\hat{y}_{t,n}^{(c)} \log p_n(c|w, I_t) + k_c \hat{y}_{t,n}^{(c)} \right], \quad k_c > 0 \quad \forall c.$$

The Step a) per-pixel solution becomes (same argument, threshold now e^{-k_c} per class):

$$\hat{y}_{t,n}^{(c)*} = \begin{cases} 1, & c = \arg \max_c \frac{p_n(c|w, I_t)}{e^{-k_c}} \text{ and } \frac{p_n(c|w, I_t)}{e^{-k_c}} > 1, \\ 0, & \text{otherwise.} \end{cases}$$

Intuition: confidence is **normalized per class** by e^{-k_c} (k_c tuned so each class admits a comparable fraction) $\rightarrow$ **balances** which classes contribute labels. Consequence: the winning class is the largest *normalized* score, so the chosen pseudo-label can **differ from the raw argmax** (a less-confident but under-represented class can win).

Selecting the thresholds (per round). Set them by a target *proportion* p of pixels to label, not by a fixed probability:

- **Basic ST:** pick k so a fraction p of *all* target pixels get a pseudo-label (e.g. p ramped $0.2 \rightarrow 0.5$ in steps of 0.05 over rounds = the curriculum).
- **Class-balanced ST:** pick k_c *per class* so a fraction p of the target pixels *whose most-probable class is c* have $p_n(c) \geq e^{-k_c} \rightarrow$ each class admits its own top- p regardless of absolute confidence.

Intuition: thresholding on a per-class **percentile** (top- p) rather than a global probability is exactly what stops easy classes from dominating.

4.1.5 Test-Time Domain Adaptation

Adapt a pretrained model *on the fly* to unlabeled test data: a **single online pass**, predicting as data streams in (no source data, no target labels, no retraining round).

Test Entropy Minimization (TENT).

Idea: modulate the network online to **minimize the entropy** of its own predictions over each input batch (confident predictions = low entropy = proxy for being well-adapted); the parameters updated by the backward pass on batch i are used in the forward pass for batch $i+1$. *Challenge:* the full network is non-linear and its parameters high-dimensional $\rightarrow$ **cannot** be effectively optimized in a single pass.

Solution: only touch BatchNorm. Optimize *just* the **batch-norm** statistics and affine parameters, because they are **linear** and act **per-channel** (cheap, stable to update online):

- **Statistics:** discard the fixed train-set running mean/var (normally frozen at inference) and instead compute mean/var **adaptively per test batch**. Side effect: an output now depends on the *other inputs* in its batch, unlike the usual fixed-linear-transform inference.
- **Parameters:** at each backward pass, update the BN **scale** γ and **shift** β of every layer via the gradient of the entropy loss. These vary dynamically across batches through testing.

Intuition: the domain shift is largely a shift in *feature statistics*; re-estimating BN stats + nudging γ, β to sharpen predictions realigns the features cheaply, without labels. A **single epoch** suffices (offline, one can run multiple epochs over the whole test set).

4.2 Sensor Fusion

Robust Perception across multiple modalities.

4.2.1 Basics and Taxonomy

- **Late Fusion:** Fuse individual *results* (e.g. predictions) from the sensors. Seperate networks
- **Early (Low-Level) Fusion:** Fuse the raw sensor data, or combining raw lidar with image features, then processed jointly through single architecture
- **Mid-Level/ Deep Fusion:** Fuse internal features from the two sensors, i.e. feature extration is seperate, yet the task backbone network is shared
- **Asymmetric Fusion:** Fuse object-level representations (i.e., results/predictions) from one sensor with feature-level or data-level representations from the other (just some fusiun module required)
- **Weak Fusion:** One sensor only for guidance (auxilliary/ condition) to the other for generating features or results (not the actual data or features used, hierarchical)

Trade-offs (early vs. deep vs. late).

- **Early:** suffers *inherent discrepancies* between modalities — alignment, representation, sparsity — fused too soon.
- **Deep:** fixes this by processing modalities *separately* first, but is **complex, sensitive to calibration**, and needs **abundant labeled multi-modal data**.
- **Late** (fuse at *object-candidate* level): (+) train modality-specific detectors on *large unimodal* data, multi-modal data only needed for the small **fusion module**; (+) boxes are simpler to process than raw multi-modal data; (→) fusion just *prunes/associates* predictions across modalities, so the unimodal detectors should be tuned for **maximum recall**.

4.2.2 Lidar-Camera Fusion for 3D Object Detection

Improve lidar 3D detection networks with camera data (high-resolution features with color texture) for better recall on distant objects and localization accuracy.

Frustum PointNets (Weak Lidar-Camera Fusion).

Camera is used as guidance to process and use the Lidar inputs. Use 2D BB (outputs from one. detection branch) to constrain/ filter out some of the lidar point cloud in the 3D search space.

These 2D detectors are very strong, yet only operate in 2D. By using the perspective projection model of the camera we can project into the 3D space (as a frustum) of the Lidar and guide 3D detection. The 3D instance sementation is performed on the frustum's point cloud as binary point cloud segmentation (PointNet: Object of interest vs Background). The segemented set of points is input to an *amodal 3D BB regression Point Net* which predicts box (residuals) that extends to the back part of the object, which is not visible by the lidar (thats amodal) — reason about the invisible parts.

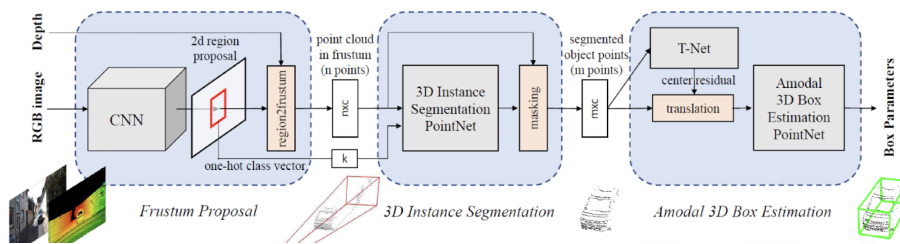


Figure 24: Frustum PointNets: weak fusion of Lidar and Camera Sensors

Coordinate normalization (key idea). Pose is factored out *before* regressing the box, via a chain of canonicalizing transforms (each undone later to recover the absolute box) — same philosophy as PointRCNN's canonical coordinate system, but in stages:

1. **Camera → frustum (rotation):** rotate so $+z$ aligns with the frustum's central axis (the 3D column derived from the camera image) → removes dependence on *where* in the image the object sits (viewing direction).
2. **Frustum → 3D mask (translation):** move the origin to the **centroid of the segmented object points** → centers the (partial) object.

3. **3D mask** $\rightarrow$ **3D object** (*translation, learned*): shift by the **T-Net**-estimated center — a small regression PointNet predicting the true object center, since the segmentation centroid is biased by partial/occluded views.

Center decomposition. The absolute 3D box center is the *sum* of three parts: (1) segmented-points centroid, (2) T-Net residual, (3) the box head's regressed center residual. *Intuition:* each transform moves into a more canonical frame, so the net only predicts small **residuals** from a normalized origin (easier/more stable), and the true center is reassembled by adding the shifts back.

Multi-View 3D Object Detection Network (MV3D Asymmetric + Deep Fusion)

Working directly in the raw 3D space is more complex if we do not want to rely on 2D box predictions. We do this by using lower-dimensional spaces (projections) than raw 3D space considering 2D (birds-eye or camera perspective views).

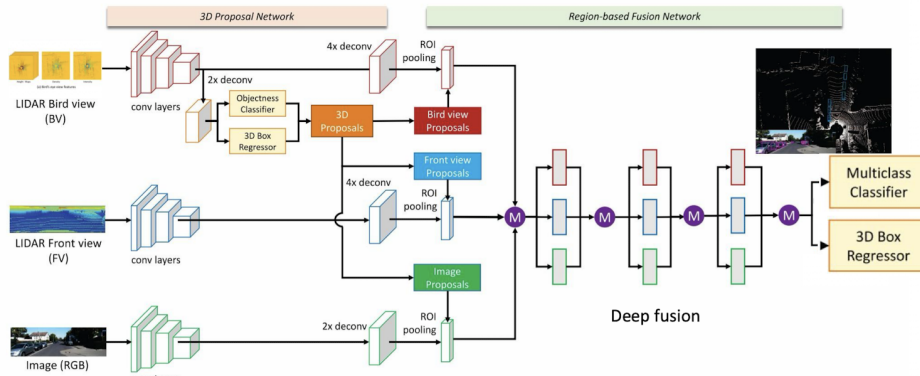


Figure 25: Multi-View 3D Object Detection Network

Asymmetric Fusion via Object Level representation fused with features from other branches, and **deep fusion** at different layers in the network (multi-sensor features).

For **driving scenes** the birds-eye view (from Lidar) is the primary modality (very decomposed and cononical repr.) using Conv layers for feature representation followed by some R-CNN approach for Object Classification and 3D Box regression.

Lidar creates a front-view (camera-like). Passed for feature repr. Same for the image data. These features are then used in *asymmetric* fusion with the 3D proposals from the BEV $\rightarrow$ becomes a ROI proposal. (RoI Pooling (asymmetric fusion impl.) - fixed length features for all modalities/ views from the three different input features) *Deep fusion* is then applied at the level of regions.

Why BEV? (vs. front/camera view).

- **Size preserved:** objects keep their physical size under BEV projection $\rightarrow$ *small size variance* (no perspective scaling) $\rightarrow$ easier for anchor-based detection.
- **No occlusion:** objects occupy *different* parts of the ground plane $\rightarrow$ they don't overlap as in a perspective view.
- **Sufficient for driving:** road-scene objects mostly *lie/stand on the ground plane*, so the dropped height dimension costs little.

Limitations: BEV is **coarse/sparse** $\rightarrow$ information lost especially for **small objects**; and it **cannot resolve multiple objects stacked along the vertical (z) dimension**.

Augmenting Lidar Point Clouds

A common issue with Lidar PointClouds is the semantic differentiation of objects, e.g. a person vs. pole at distance, due to low vertical resolution. We augment these points with semantic features.

PointPainting (Early Fusion): Paint the point clouds (augment via concatenation of the lidar point x,y,z , intensity plus the feature info) based on the segmentation results (softmax scores) of an image based method. Using these painted Point Clouds (not voxelized) we can then apply Lidar Detectors (3D):

$$T = T_{(\text{camera} \leftarrow \text{ego})} T_{(\text{ego}_{t_c} \leftarrow \text{ego}_{t_l})} T_{(\text{ego} \leftarrow \text{lidar})} \quad \text{lidar} \rightarrow \text{camera projection chain for sensor fusion}$$

MVX-Net (Multimodal VoxelNet): like PointPainting, but fuses *2D-detector* features (VGG16 **conv5** from a Faster-RCNN backbone) instead of semantic segmentation features so we now use voxelized representations. Two variants:

- **Early fusion**: project each point to the image, *index* its conv5 feature, dim-reduce, **point-wise concat** with $(x, y, z, r) \rightarrow \text{VFE} \rightarrow \text{3D RPN}$ (same as PointPainting, different features).
- **Mid-level (voxel) fusion**: project each *non-empty voxel* onto the image $\rightarrow$ **ROI-pool** the conv5 features in that region $\rightarrow$ concat with the VFE-encoded voxel feature. Fuses deeper, can include empty voxels, and is **more memory-efficient** (fewer features per point in the VFE).

Which image features to fuse?

- **Segmentation softmax** (PointPainting): informative for *classification* but *not* for box localization; pixel-level semantic labels are usually **unavailable** in 3D-detection datasets $\rightarrow$ unreliable (missing supervision).
- **2D-detection features**: learnt **reliably** (abundant GT from the 3D box annotations) and more *relevant* to 3D detection (similar tasks). But MVX-Net’s conv5 is high-level/global $\rightarrow$ **poorly localized**.

PointAugmenting (Early & Mid-level Fusion): fixes the localization issue by using **mid-level** 2D detection features (preserve fine spatial detail).

- **Early**: fetch image features for the corresponding lidar points (point-wise), processed *separately* from the lidar features by the voxel encoder.
- **Mid-level**: fuse *inside* the 3D backbone — after the voxel grid is reduced to a 2D BEV map, before the RPN.

Limitations of point-level fusion: (i) simple **addition/concatenation** of lidar + image features $\rightarrow$ performance *deteriorates* when image features are degraded (e.g. low illumination); (ii) **hard association** (one pixel per point) discards most of the rich image features around the projection and is **sensitive to extrinsic calibration & lidar-camera synchronization** (prone to misalignment).

TransFusion (Mid-level Fusion with Soft Association)

Replaces the brittle *hard* association by a **soft association** of lidar and image features via **cross-attention**: the network *implicitly* learns which image features to attend to (and which to ignore) per object $\rightarrow$ **robust** to degraded images and calibration/sync errors. A transformer detector with two decoder layers: queries attend first to **LiDAR BEV** features $(K, V) \rightarrow$ initial prediction, then to **image** features $(K, V) \rightarrow$ final output. Unlike DETR, queries are **input-dependent** and **image-guided** (not fixed/learned).

Query initialization. Predict a *class-specific* BEV heatmap $\hat{S} \in \mathbb{R}^{X \times Y \times K}$, run **NMS**, take the **top- N** peaks as queries (positions + features) $\rightarrow$ queries already sit on likely objects (fast convergence vs. DETR’s random queries).

Image guidance. **Collapse image features along the height axis** (max-pool, since ≤ 1 object per image column) $\rightarrow$ cheap, little info lost; use them as K, V in multi-head attention with LiDAR BEV as $Q \rightarrow$ **fused BEV features** that hint at object positions, improving query init.

SMCA — spatially modulated cross-attention. *Problem*: lidar and image come from *different domains*, so queries may attend to image regions unrelated to the box $\rightarrow$ slow training. *Fix*: softly constrain attention to a 2D region around the predicted center (do not attend blindly): (1) project the query’s BEV position to the camera view (c_x, c_y) ; (2) multiply the attention map element-wise by a soft Gaussian mask

$$M_{ij} = \exp\left(-\frac{(i-c_x)^2 + (j-c_y)^2}{\sigma^2}\right).$$

Intuition: focuses on foreground pixels near the object center, ignoring background $\rightarrow$ cleaner semantics for classification + box regression, faster to train.

Camera-Lidar Object Candidates.

CLOCs = late fusion that *refines 3D-detection confidences* using 2D detections. Works on *candidates* (boxes + scores), so it combines with **any** 2D and 3D detector that outputs confidence — no joint backbone training.

Inputs. k 2D detections P^{2D} with $p_i^{2D} = [x_{i1}, y_{i1}, x_{i2}, y_{i2}, s_i^{2D}]$, and n 3D detections P^{3D} with $p_j^{3D} = [h, w, l, x, y, z, \theta, s_j^{3D}]$ (boxes + single-class confidence).

Association tensor. Project each 3D box to the image and compute $\text{IoU}_{i,j}$ with every 2D box (*geometric consistency* of a cross-modal pair). Build a **sparse** tensor $\mathbf{T} \in \mathbb{R}^{k \times n \times 4}$ filled *only* where $\text{IoU}_{i,j} > 0$:

$$\mathbf{T}_{i,j} = \{\text{IoU}_{i,j}, s_i^{2D}, s_j^{3D}, d_j\},$$

with d_j the lidar-origin-to-box-center distance in the horizontal plane, normalized by lidar range. A 3D box with *no* 2D overlap gets $\text{IoU} = s^{2D} = -1$ (encodes "unmatched").

Fusion network. Gather the p non-empty entries $\rightarrow (1 \times p \times 4)$, pass through 1×1 convs $\rightarrow (1 \times p \times 1)$, scatter back into $\mathbf{T}_{out} \in \mathbb{R}^{k \times n \times 1}$ by index, **max-pool over the k (2D) axis** $\rightarrow$ squeeze $\rightarrow n$ **refined scores** (one per 3D detection). Trained with **focal loss**.

Ablation (KITTI car). Every channel + focal loss contributes; full config $\rightarrow$ 3D AP **82.73** (BEV 92.57) vs. SECOND baseline 79.94. The IoU (geometric cue) and the 3D score s^{3D} matter most.

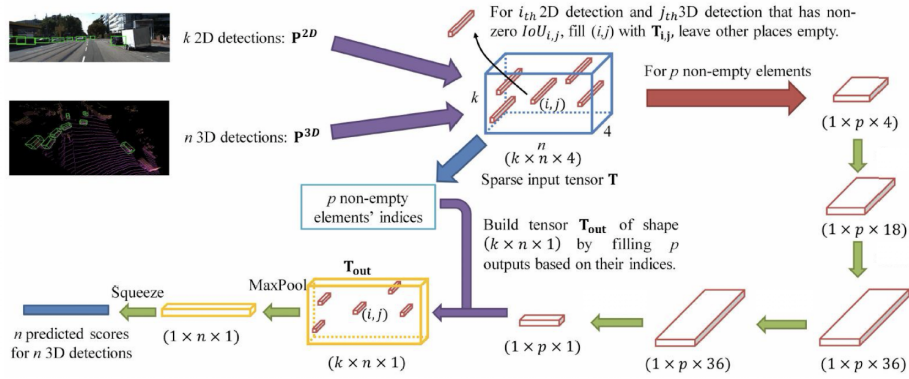


Figure 26: Camera-Lidar Object Candidates

4.2.3 Radar-Camera Fusion for 3D Object Detection

Unlike lidar-camera (which improves the already-precise *lidar* 3D detection with camera), radar gives only **coarse 2D azimuth-range** measurements (sparse points or a dense BEV map), *not* precise 3D $\rightarrow$ radar **cannot be the primary sensor**. So radar-camera fusion improves **camera-based** 3D detection with radar as an *additional* modality; the two are **complementary** — camera gives semantics + lateral position, radar gives **depth/range** and (Doppler) **velocity**.

CenterFusion (Asymmetric Radar-Camera Fusion).

Built on **CenterNet** (objects as center points). Pipeline:

1. **Image $\rightarrow$ primary heads:** FCN backbone $\rightarrow$ center heatmap (HM) + primary regressions (offset, WH, dimension, depth, rotation) $\rightarrow$ a *preliminary* 3D box per object.
2. **Frustum association** (the *asymmetric* step): use each preliminary box — *object-level*, from the camera — to associate *feature-level* radar data (see below).
3. **Concat + secondary heads:** concatenate the associated radar features with image features $\rightarrow$ secondary regressions refine **depth, velocity, rotation, attributes**.
4. **3D box decoder:** takes depth/velocity/rotation/attributes from the *secondary* heads, all other properties from the *primary* heads.

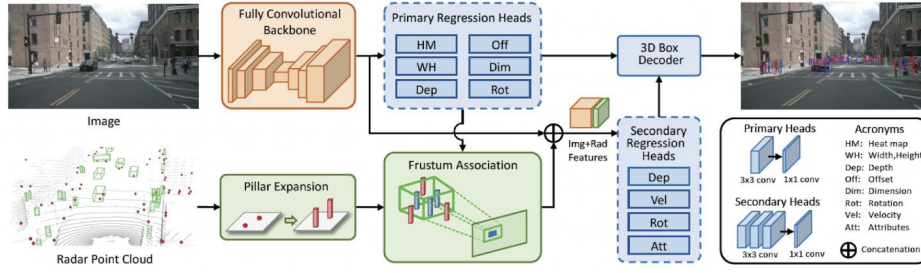


Figure 27: CenterFusion

Frustum-based radar association. Radar is sparse and *lacks the vertical dimension*, so: (i) expand each radar point into a fixed-size **vertical pillar** (accounts for missing/inaccurate height); (ii) associate a pillar with an object if (part of) it lies inside the object's **ROI frustum** (built from the preliminary 3D box); (iii) if several radar detections match, take the **closest**. (A margin δ enlarges the frustum at test time.)

Radar feature extraction. For each associated object, generate radar **heatmap channels** — radial velocity in x , y and **depth** — placed in a region around the object center scaled by its box size ($F_{x,y,i}^j = \frac{1}{M_i} f_i$ inside $|x - c_x^j| \leq \alpha w^j$, $|y - c_y^j| \leq \alpha h^j$, else 0), then **concatenated with the image features**. Overlap rule: where two objects' heatmaps overlap, the **smaller-depth (closer)** object dominates (only the closest is fully visible).

4.2.4 Multi-Model 2D Object Detection

Why 2D detection at all? Many salient objects are *missing from the lidar point cloud* (e.g. **42%** of objects in DENSE's "dense fog" split get only 2D, not 3D, annotations). 2D boxes still give **class, location, size** — useful for downstream prediction/planning. So fuse *all* sensors with the basic camera modality for 2D tasks too.

Why fuse for 2D? Cameras are **passive** → degrade badly in **low illumination** (night) or **high dynamic range**. Other sensors are **complementary**:

- **Lidar:** robust to illumination, rich 3D structure, but sensitive to particles (fog/rain/snow).
- **Radar:** robust to illumination *and* weather, but coarse 2D readings + noisy.

Rationale: full autonomy needs robust perception under **all** visual conditions → no single modality suffices.

CRF-Net (Camera-Radar Fusion for 2D Detection)

Challenge — association. 2D radar has *no height* (vertical) info. *Fix:* extend each radar point into a **vertical pillar** spanning 3m from the ground plane (covers an object's vertical extent), project pillars to the image as **1-pixel-wide lines**, and build a **radar image** whose channels paint each pillar with **radar distance & cross-section** (pixels outside any pillar = 0).

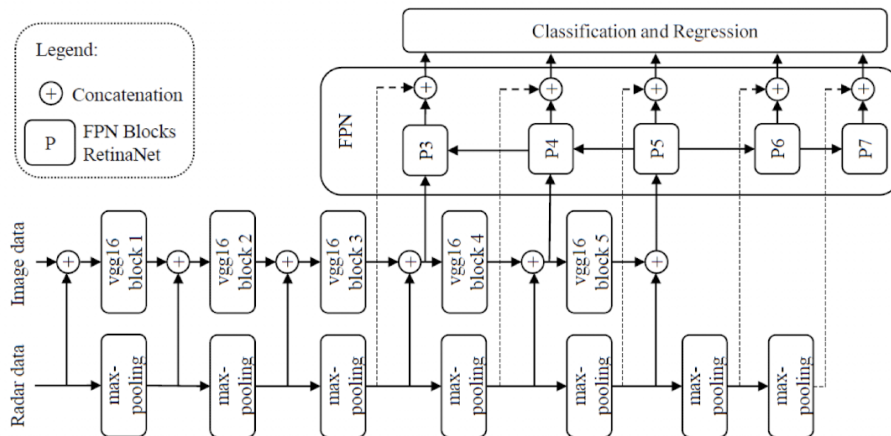


Figure 28: CRFNET

Multi-level fusion (RetinaNet + FPN backbone). Radar distance/cross-section at a pixel is *more* informative than a plain RGB value → **early fusion is suboptimal**, but the right abstraction level for mid-fusion is unclear. *Solution:* **fuse at multiple deep layers** (concatenate radar + image feature maps at several VGG blocks) and let the network *learn where* to fuse most strongly.

BlackIn training. The net is pretrained *image-only*; to force it to actually use radar (and depart from the camera-only optimum), **randomly deactivate the camera-input neurons** per batch with fixed prob. (0.2) — a dropout-on-one-modality trick that pushes reliance onto the radar branch.

Robust Sensor Fusion

Problem: previous methods assume sensor streams are **consistent & redundant**, but adverse weather/illumination cause **asymmetric failures** (one modality degrades, others don't). Need to **fuse adaptively** — across samples and within a sample — learning to **ignore** noisy/degraded (regions of) measurements and **rely on** high-SNR ones. Two solutions:

- **Entropy-based fusion:** trust the lower-entropy (more confident) sensor/region. *Caveat:* "high entropy ⇒ low SNR" does *not* always hold — e.g. lidar in snowfall (spurious confident clutter).
- **Attention-based fusion:** learn the per-sensor/per-region weight **implicitly from data** via **cross-attention** (as in TransFusion) — no hand-set rule.

Scaling issue. Naive pairwise feature exchange scales **quadratically** with the number of modalities (memory & compute). Cross-attention only handles **two streams** — a *main* stream (queries) and an *auxiliary* one (keys/values) — so the open question is how to apply it efficiently to an **arbitrary number** of modalities.

HRFuser answers this: a **modular** multi-resolution architecture (generalizes **HRNet** to multiple modalities) with **camera as primary** and extra sensors as additional branches. Fusion uses **multi-window cross-attention (MWCA)** blocks whose **parallel cross-attention** (camera as query, each extra sensor as keys/values, run in parallel) scales to an **arbitrary number** of sensors — avoiding the quadratic blow-up. Three design choices drive its **efficiency**: (i) the lower-resolution, higher-dimension feature maps are computed *only* for the primary camera modality (not the others); (ii) parallel cross-attention keeps the non-primary modalities from attending to *each other* (only camera↔sensor, never sensor↔sensor); (iii) cross-attention is applied within *multiple disjoint spatial windows*.

4.3 Visual Grounding, Anomaly Segmentation, and Vehicle-to-Vehicle Communication

4.3.1 Visual Grounding

Motivation: an ADS carries **passengers** who may give **natural-language instructions** tied to the driving task (e.g. "park behind the red car on the right"), to be processed in *real time* → **verbo-visual perception**.

Definition. Given (1) a visual scene and (2) a **verbal description** of an object in it, **detect the referred object** — i.e. "**ground**" the description in the scene (a.k.a. **referral-based object detection**). Variants by input: **2D** (image) vs. **3D** (colored point cloud).

3D visual grounding — task variants.

- **Referral-based 3D localization:** detect the referred object *directly* from the point cloud.
- **Referral-based 3D identification:** *classify* which of the *given* boxes (all objects of the referred class) is the referred one.

Localization is harder than identification: it must **detect** multiple candidates across *several* classes (not just the referred class) and **discriminate** the referred object from all other candidates.

Grounding-by-detection framework. Two backbones — a **language** backbone (word embeddings) and a **visual 3D** backbone → **proposal generation** — feed a **verbo-visual fusion** module that outputs per-proposal **confidence scores**; box regression / mask generation then yields the grounded box or instance mask.

3DVG-T (Verbo-Visual Fusion via Transformers).

Fuse via **cross-attention** between L word embeddings and M 3D proposal embeddings (same dimensionality). **Interlaced** self- and cross-attention layers, several pairs **cascaded** → information flows both *between proposals* (self-attn) and *across proposals↔words* (cross-attn) → object confidence scores.

Challenge in 3D — View Dependency.

Relational **spatial specifications** ("on the left of") *depend on the annotator's viewpoint*, which is *dynamic* during annotation → **ambiguity** in the language→object association. Relevant for AD: passengers have varying viewing directions. Two solutions below.

MVT-3DVG (Multi-View Transformer).

Idea: make grounding **view-robust** by aggregating over **multiple views**. Generate N rotated versions of the cloud about the **vertical axis**, $S^j = R(\theta_v^j) S$ with $\theta_v^j = \frac{2\pi(j-1)}{N}$, and aggregate with a **permutation-invariant** g : $VG(S, Q) = g(f(S^1, Q), \dots, f(S^N, Q))$.

Problem: processing N clouds end-to-end is **expensive**. *Fix — decouple*: (i) point-cloud features (affine-**invariant**) are extracted **once per object** ($x_i = \text{LN}(W_x \text{PointNet}++(pc_i))$) and **shared** across views; (ii) only object *centers* are rotated per view to give view-specific **positional** embeddings $\text{PE}(b_i^j)$; combine $o_i^j = x_i + \text{PE}(b_i^j)$.

ConcreteNet (Learned Camera Embedding).

Idea: resolve view dependency with a **learned camera embedding** inside attentive verbo-visual fusion, supervised on the predicted **camera position** $\mathbf{t}$ at train time ($\mathcal{L}_{\text{camera}} = \|\mathbf{t} - \mathbf{t}^*\|$). *Why only position*: the camera always looks *at* the scene and higher scene parts map to higher pixels, so rotation (look-at, roll) adds little signal. Two more components:

- **Bottom-up attentive fusion**: 3D masked attention with **spherical masks** $\mathbf{M}_l$ of *increasing* radius r_l to progressively aggregate wider context ($\mathbf{M}_l(i, j) = 0$ if $\|o_i - o_j\| < r_l$, else $-\infty$).
- **Verbo-visual contrast**: instance-level contrastive loss (NT-Xent) with the *language* embedding as anchor, the *true* referred instance as positive, all other instances as negatives → structures a joint embedding space.

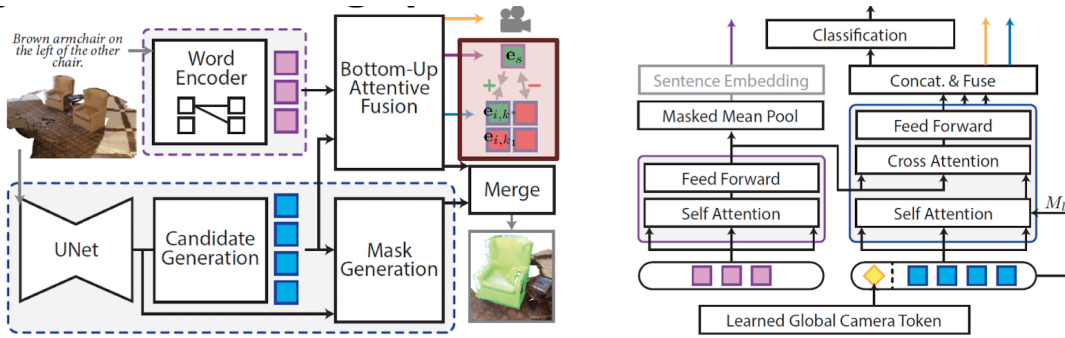


Figure 29: ConcreteNet

4.3.2 Anomaly Segmentation

Goal (outlier-aware perception): flag **out-of-distribution (OOD)** pixels — objects of *unknown* classes that standard segmentation silently mislabels as a known class. Predict a **soft dense anomaly map** in $[0, 1]$.

Evaluation & inference. Threshold the map at many values → precision/recall → **AP** (overall metric); **FPR95** (false-positive rate at 95% recall) matters for safety. At inference: either fix a threshold for **hard anomaly masks** (combined with in-distribution predictions), or keep the **soft map** as uncertainty for downstream planning.

Training regime. Anomalies are OOD → *cannot* be assumed seen at training. Two families:

- **Discriminative**: train *only* on in-distribution data, condition the model to expose uncertainty.
- **Generative**: additionally train on **auxiliary negative data** (typically synthetic) — still no access to the real test-time outlier distribution.

Bayesian DeepLab (Discriminative — Uncertainty).

Decompose total uncertainty into **epistemic** (model) + **aleatoric** (data); anomalies ↔ high **epistemic** uncertainty. A single deterministic pass can't estimate it → **MC dropout**: T forward passes with randomly dropped neurons.

- **Total** = predictive entropy of the mean output, $\mathbb{H}(\mathbf{y}|\mathbf{x}) = \sum_c -(\frac{1}{T} \sum_t y_c^t) \log(\frac{1}{T} \sum_t y_c^t)$.
- **Epistemic** = mutual information = total − *aleatoric* (mean per-pass entropy $\frac{1}{T} \sum_t \sum_c -y_c^t \log y_c^t$); **high** ⇒ outlier.

Image Resynthesis (Generative).

Idea: a segmentation net trained on in-distribution data labels an outlier as some *known* class; **resynthesize** the image from those predicted labels with a **conditional GAN** (trained on paired in-distribution image+labels) → **anomalous regions are reconstructed poorly** (their labels are wrong). Detect the mismatch:

- **Discrepancy network:** compares input image, predicted labels, and resynthesized image. A naive input–resynthesis pixel difference fails (the GAN globally shifts appearance).
- **Training data** (no real outliers needed): take a GT label map, *replace some instances with random labels*, resynthesize, and train the detector to recover the altered pixels.
- **Architecture: three-stream** (input & resynthesized images via shared-weight VGG16, labels via CNN), U-Net multi-resolution; feed **correlations** between input and resynthesized features to the decoder — *correlation is low in anomalous regions*.

4.3.3 Vehicle-to-Vehicle Communication for Perception

Motivation: single-car perception fails on **heavily occluded** / **far** objects (sparse observations). Multiple AVs in the **same area** can **share** perception → **see behind occlusions** & extend range. Increasingly relevant as adoption grows (AVs no longer sparse).

V2VNet (Multi-Agent Deep Fusion).

Each car: point cloud → LiDAR conv → **compressor** → **broadcast** a message; on receipt → **decompressor** → **cross-vehicle aggregation** → output net. A message carries $(\Delta t, \Delta p, \hat{z})$: **time delay**, **relative pose**, and an **intermediate representation**. Practical constraints: **limited bandwidth** (maximize message informativeness under budget) and **time delay** / **no synchronization** / **changing relative pose** across vehicles.

What to transmit? (early vs. deep vs. late).

- **Raw measurements** (early): minimal info loss, but **large volume** + expensive processing on each receiver.
- **Perception outputs** (late, e.g. 3D boxes): **tiny volume**, but no scene context/features or uncertainty.
- **Intermediate representation** (deep, V2VNet’s choice): relieves the receiver of most processing, densely captures informative features, and is **amenable to learnable compression** — best trade-off.

Results. V2VNet’s deep fusion **beats** no/LiDAR/output fusion in 3D detection, especially for **sparsely-covered** objects; with learned compression (*Ballé*) it reaches near-full AP at **~0.02 MB** (vs. 10 MB raw); and it is **robust to transmission time delays** (output/LiDAR fusion degrade). Qualitatively: detects **occluded** cars and extends **effective range**.

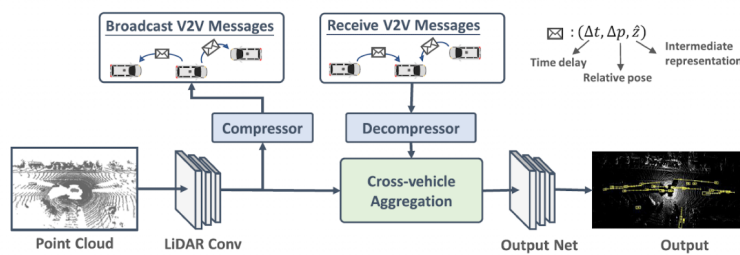


Figure 30: V2VNet

5 Part V: Temporal Perception

5.1 Multiple Object Tracking

Definition (2D monocular MOT). In an input video, **detect** all appearances of each object (predefined classes) across frames and **group** the detections of each object into a distinct set, by assigning the same unique **ID** to all of them.

5.1.1 Tracking by Detection (Classical)

The dominant classical paradigm; detection is off-the-shelf (e.g. Faster R-CNN; datasets even provide *public detections*), so research focuses on **feature extraction** and **data association**. Pipeline:

1. **Object detection** per frame.
2. **Feature extraction** (appearance + motion) per detection.
3. **Affinity computation** between detections.
4. **Association** of the detections that pertain to the same object.

Baseline.

(1) per-frame detector; (2) motion feature via a **Kalman filter** predicting boxes across consecutive frames; (3) data association as optimal **bipartite matching** between predicted boxes (from previous frames) and detected boxes (current frame), e.g. the **Hungarian algorithm** (same setup as DETR’s matching).

MOT Metrics.

(1) **Error-type metrics:** **MT** (mostly tracked: GT trajectories correct in $\geq 80\%$ of frames), **Fragments** (hypotheses covering $\leq 80\%$ of a GT trajectory), **ML** (mostly lost: correct in $< 20\%$), **False trajectories** (no real object), **ID switches** (object tracked but its ID erroneously changes).

(2) **Summarizing — MOTA.** Match object o_i & hypothesis h_j by IoU, with *continuity* (a match carried from $t - 1$ is prioritized even over a higher-IoU match: if matched at $t - 1$ and $\text{IoU} \geq 0.5$ at t , keep it); remaining matched by 0.5-IoU. With FN, FP, ID switches (IDSW):

$$\text{MOTA} = 1 - \frac{\text{FN} + \text{FP} + \text{IDSW}}{\text{GT}} \in (-\infty, 1].$$

Siamese CNNs for Feature Extraction & Affinity.

Idea: learn deep features that **classify whether a pair of detections is the same object**. Positive pairs = same GT track at limited temporal distance; negatives = different tracks, or true+false-positive, or two false positives (FP boxes sampled randomly without overlap with true detections). Input: **appearance** I + **optical flow** O (5 channels per detection, fixed size, **Siamese junction at the start**). Siamese deep features + **contextual features** (relative size change, position change, velocity) $\rightarrow$ **gradient-boosting** / **random-forest** classifier $\rightarrow$ matching score in $[0, 1]$. (Developed for multi-pedestrian, a well-defined class.)

Data Association via Linear Programming (Max-Flow).

Detections $\mathcal{D} = \{\mathbf{d}_i^t = (x, y, t)\}$; a trajectory $T_k = \{\mathbf{d}_{k_1}^{t_1}, \dots\}$ is an ordered list of detections. Find the set of trajectories $\mathcal{T}^*$ that best explains the detections, as a **network-flow LP**:

$$\mathcal{T}^* = \arg \min_{\mathcal{T}} \sum_i C_{\text{in}}(i) f_{\text{in}}(i) + \sum_i C_{\text{out}}(i) f_{\text{out}}(i) + \sum_i C_{\text{det}}(i) f(i) + \sum_{i,j} C_t(i, j) f(i, j),$$

with **flow-conservation** and **exclusion** constraints; link costs C_t come from the **Siamese affinity**. *Why the signs:* in/out costs are **positive** (else the tracker spawns too many trajectories), while detection/link costs are **negative** (else zero flow = no trajectory would be the trivial optimum).

Limitations of Tracking by Detection.

Association is a mere **post-processing** step on heuristic similarities; **detection is single-frame** and decoupled from association; **temporal modeling is minimal**.

5.1.2 End-to-End MOT with Transformers (MOTR)

Inspiration — *DETR*: detection = predicting a **set** of objects. **MOTR**: MOT = predicting a **set of object sequences** (each sequence = a track). Two query types:

- **Detect queries**: detect **newborn** objects (standard DETR object queries, *fixed* set).
- **Track queries**: re-detect objects **continuing from previous frames**; this set has **variable length**.

Tracklet-aware label assignment. *Propagate* track-query assignments directly from the previous frame (each tracked object keeps **consistent** GT boxes); run **bipartite matching only between newborn GT objects and detect queries**.

Architecture. Per frame: Enc $\rightarrow$ Dec fed with concatenated detect queries q_d + track queries q_{tr} ; a **Query Interaction Module (QIM)** updates the track queries for the next frame. In the decoder self-attention, **detect queries are discouraged** from focusing on already-tracked objects (interaction with track queries). Training: aggregate single-frame DETR losses over whole clips.

Temporal interaction (QIM). Handles object **entrance** (filter newborns by score) and **exit** (drop disappeared tracks); a **TAN** (Temporal Aggregation Network = MHA over track queries) refines them. Entrance/exit: by GT assignment at training, by score **thresholding** at inference.

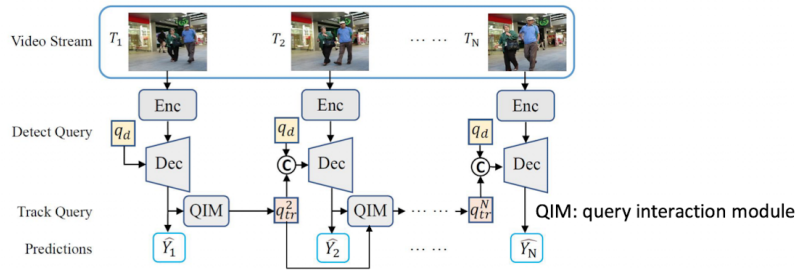


Figure 31: MOTR

5.2 Motion Prediction

Goal (forecasting): to plan a trajectory in a dynamic environment we must know **where obstacles will be**; key for **trajectory planning**. Typically in **BEV** space, from detections or tracks.

Classical baselines.

- **Constant velocity**: keep speed & direction. Good *short-term*; no traffic rules, no social interactions.
- **Lane-network based**: associate the car with a lane and follow it. Adds *some rules* & enables **multimodal** predictions; still no social interactions, velocity must be estimated.

Why social interactions matter: motion strongly depends on *interactions between drivers*, who **don't always follow rules** (e.g. a car stops to let another pass) $\rightarrow$ learn these interactions from data.

CNN-based prediction.

Render the traffic situation with the car of interest **centered**; a CNN encodes it and an **FC** or **RNN** head predicts the trajectory $((X_1, Y_1), \dots, (X_N, Y_N))$. Handling temporal data: render **past** into frames; optionally add **past state** (v, a) sequence in **late fusion** (separate FC); or feed **multiple frames** to a 2D-3D CNN. *Limitation*: predicts **one agent** at a time.

RNN-based prediction.

Directly use the **positions** of all cars, encode each sequence with an **LSTM encoder**, decode the future trajectory with an **LSTM decoder** (per agent). Model **social interaction** with a dedicated module (**social pooling / attention**) inserted *between* encoders and decoders $\rightarrow$ **simultaneous multi-agent** forecasting. Learned attention heads attend e.g. to the front vehicle in the same lane, or the closest front vehicle from any lane.